

ampliar un diagrama, para mostrar los elementos anidados, para atravesar enlaces a otros modelos o vistas; y animación. Sería imposible intentar estandarizar todas estas posibilidades y absurdo intentarlo, porque no hay necesidad y limitaría la innovación útil. Esta clase de extensiones de notación es responsabilidad del constructor de la herramienta. En una herramienta interactiva hay menos peligro de ambigüedad, porque el usuario puede pedir siempre una aclaración. Esto es probablemente más útil que insistir en una notación que sea totalmente inequívoca en el primer vistazo. Lo importante es que una herramienta debe poder producir la notación canónica cuando ésta sea solicitada, especialmente en forma impresa, y que una herramienta interactiva debe suministrar extensiones razonables a este comportamiento mínimo.

Contamos con que las herramientas también permitan que los usuarios extiendan la notación de maneras limitadas pero útiles. Hemos especificado que los estereotipos pueden tener sus propios iconos. Podrían estar permitidas otras clases de extensiones de notación, pero los usuarios deben aplicar un buen criterio.

Observe que la notación es algo más que figuras; incluye la información en forma de texto y los hiperenlaces invisibles entre elementos de representación.

Responsabilidades del lenguaje de programación

UML debe trabajar con varios lenguajes de implementación, sin incorporarlos explícitamente. Pensamos que UML debe permitir el uso de cualquier (o por lo menos de muchos) lenguajes de programación, para la especificación y la generación del código final. El problema es que cada lenguaje de programación tiene muchos elementos semánticos, que no deseamos absorber en UML, porque se manejan mejor como elementos del lenguaje de programación, y hay variaciones considerables en la semántica de ejecución. Por ejemplo, la semántica de concurrencia se maneja de maneras diferentes entre los lenguajes (si es que se maneja en todos).

Los tipos de datos primitivos no se describen detalladamente en UML. Esto es deliberado pues no deseábamos incorporar la semántica de un lenguaje de programación en preferencia a todos los demás. Para la mayoría de los propósitos de modelado, esto no es un problema. Utilice el modelo semántico aplicable a su lenguaje. Éste es un ejemplo de un punto de variación semántico.

La representación de las propiedades detalladas del lenguaje para la implementación suscita el problema de capturar la información sobre propiedades de la implementación sin la construcción de su semántica en UML. La decisión tomada fue capturar las propiedades del lenguaje que van más allá de capacidades incorporadas en UML, por medio de estereotipos y de valores etiquetados. Éstos se pueden asignar a las propiedades del lenguaje y a las opciones de la generación de código por una herramienta o un generador de código. Un editor genérico no necesita entenderlos. Un usuario podría, de hecho, crear un modelo usando una herramienta que no contemple el lenguaje a utilizar, y transferir el modelo final a otra herramienta para el procesamiento final. Por supuesto, si la herramienta no entiende los estereotipos y las etiquetas, no puede comprobarlas para saber si hay consistencia. Pero esto no es peor que la práctica normal con los editores de textos y los compiladores. En caso de necesidad, se puede crear una herramienta que utilice un conjunto particular de extensiones.

La generación de código y la ingeniería inversa requerirán, al menos en el futuro próximo, la participación del diseñador además de un modelo de UML. Las directivas y ayudas al generador de código se pueden proveer como valores etiquetados y estereotipos. Por ejemplo, el modelador

podría indicar qué tipo de clase contenedora se debe utilizar para implementar una asociación. Por supuesto, esto significa que los ajustes para generación de código en las herramientas pueden ser incompatibles, pero no creemos que haya actualmente suficiente acuerdo sobre la forma correcta de estandarizar los ajustes reales. En cualquier caso, diversas herramientas utilizarán sus generadores de código como su ventaja competitiva. Eventualmente, los ajustes por defecto pueden emerger y llegar a estar maduros para la estandarización.

Modelado con herramientas

Para modelar sistemas de tamaño real se necesitan herramientas. Las herramientas proporcionan maneras interactivas de ver y editar modelos. Proporcionan un nivel de organización, que está fuera del alcance del UML por sí mismo, pero que permite entender al usuario y ayuda en el acceso a la información. Las herramientas ayudan a encontrar la información en modelos grandes, buscando y filtrando lo que se presenta.

Algunos aspectos sobre el uso de herramientas

Las herramientas se ocupan de la organización y del almacenamiento físico de los modelos. Deben dar soporte múltiples equipos de trabajo en un sólo proyecto, así como permitir la reutilización entre proyectos. Los siguientes elementos están fuera del alcance del UML canónico, pero se deben considerar para el uso real de la herramienta.

Ambigüedades e información sin especificar. En las etapas iniciales faltan aún muchas cosas por decir. Las herramientas deben poder ajustar la precisión de un modelo y no forzar cada valor para ser específico. Véanse las siguientes secciones “Modelos inconsistentes para el trabajo en marcha” y “Valores sin especificar y nulos.”

Opciones de presentación. Los usuarios no desean ver toda la información todo el tiempo. Las herramientas deben permitir filtrar y ocultar la información no deseada en un momento dado, así como visualizaciones alternativas, usando las capacidades de visualización del hardware disponible. Esto se ha descrito ya, en la sección “Responsabilidades de la Notación.”

Gestión del modelo. El control de configuración, el control de acceso, y la gestión de versiones de las unidades del modelo está fuera del alcance de UML, pero son cruciales para el proceso de ingeniería del software y están en la cima del metamodelo.

Interfaces con otras herramientas. Los modelos necesitan ser manejados por los generadores de código, las calculadoras de métricas, los escritores de informes, los motores de ejecución, y otras herramientas de base. La información para otras herramientas necesita ser incluida en los modelos, pero no es información de UML. Los valores etiquetados son apropiados para contener esta información.

Modelos inconsistentes para el trabajo en marcha

La última meta del modelado es producir una descripción de un sistema en un cierto nivel de detalle. El modelo final debe satisfacer varias restricciones de validación para ser significativo. Sin embargo, como en todo proceso creativo, el resultado no se produce necesariamente de forma lineal. Los productos intermedios no satisfarán todas las restricciones de validez en cada paso. En la práctica, una herramienta debe manejar no sólo semánticamente los modelos válidos

que satisfacen las restricciones de validez, sino también los modelos sintácticamente válidos, que satisfacen ciertas reglas de construcción pero pueden violar algunas restricciones de validez. Los modelos semánticamente inválidos no son directamente utilizables. En su lugar pueden considerarse como “trabajos en marcha”, que representan las trayectorias hacia el resultado final.

Valores sin especificar y nulos

Un modelo completo debe tener valores para todos los atributos de sus elementos. En muchos casos, *null o nulo* (ningún valor) es uno de los valores posibles, pero si un valor puede ser nulo o no, debe ser parte de la descripción del tipo del atributo; muchos tipos no tienen un valor nulo natural dentro de su gama de valores. Por ejemplo, nulo no tiene ningún sentido como límite superior en el tamaño de un conjunto. O el conjunto tiene un tamaño superior fijo o no hay límite, en cuyo caso su tamaño máximo es ilimitado, así que la posibilidad de ser nulo es un añadido al rango de posibles valores de un tipo de datos.

Por otra parte, durante las primeras etapas del diseño, un desarrollador puede no cuidar el valor de una propiedad en concreto. Puede ser que sea un valor que no es significativo en una etapa particular, por ejemplo, la visibilidad al hacer un modelo del dominio. O el valor puede ser significativo pero el modelador pudo no haberlo especificado todavía, y el desarrollador necesita recordar que aún es necesario elegir su valor. En este caso, el valor está sin especificar. Esto indica que un valor será eventualmente necesario, pero que todavía no se ha especificado. No es igual que un valor nulo, que puede ser un valor legítimo en el modelo final. En muchos casos, particularmente con las cadenas, un valor nulo es una buena manera de indicar un valor sin especificar, pero no son lo mismo. Un valor sin especificar no es significativo en un modelo bien formado. La definición de UML no maneja valores sin especificar. Son responsabilidad de las herramientas que soportan UML, y se consideran parte de un trabajo en marcha que, por necesidad, no tiene ningún significado semántico.

Parte 3: Referencia



abstracción

1. Acto de identificar aquellas características esenciales de una cosa que la distinguen del resto de cosas. La abstracción implica buscar parecidos en conjuntos de cosas, centrándose en las características esenciales comunes. Una abstracción siempre implica la perspectiva de quien la realiza, por lo que pueden existir diferentes abstracciones para la misma cosa según el propósito. Todo modelado implica abstracción, a menudo en diferentes niveles para propósitos distintos.
2. Tipo de dependencia que relaciona dos elementos que representan el mismo concepto en diferentes niveles de abstracción.

Véase derivación, realización, refinamiento, traza.

Semántica

Una dependencia de abstracción es una relación entre dos elementos que se encuentran en niveles distintos de abstracción, como las representaciones en diferentes modelos, en niveles distintos de precisión, en niveles distintos de concreción o en niveles distintos de optimización. Generalmente las dos representaciones no se utilizarán simultáneamente. Normalmente un elemento es más detallado que otro, donde el más detallado es el cliente y el otro el proveedor. Si no está claro qué elemento es más detallado, se puede modelar cualquiera de los dos elementos como cliente.

Los estereotipos de dependencia de abstracción son la traza (palabra clave «**trace**»), el refinamiento (palabra clave «**refine**»), la realización (palabra clave «**realize**») y la derivación (palabra clave «**derive**»).

Notación

Una dependencia de abstracción se representa mediante una flecha con línea discontinua que va del elemento cliente al elemento proveedor etiquetada con alguna de las palabras clave «**trace**», «**refine**» o «**derive**». La dependencia de realización tiene su propia notación, que consiste en una flecha con línea discontinua cuyo extremo es un triángulo cerrado que apunta al elemento proveedor.

La correspondencia entre elementos se puede adjuntar a la relación como restricción.

Elementos estándar

derive, refine, trace

abstracto/a

Clase, caso de uso, señal, otro clasificador u otro elemento generalizable que no puede ser instanciado directamente. También se utiliza para describir una operación que no tiene implementación. Antónimo: concreto/a.

Véase operación abstracta, elemento generalizable.

Semántica

Una clase abstracta es una clase no instanciable, es decir, que no puede tener instancias directas, bien porque su descripción es incompleta (le falta el método de una o más operaciones) o porque no ha sido pensada para ser instanciada aunque su descripción esté completa. El objetivo fundamental de las clases abstractas es la especialización. Para ser útil, una clase abstracta debe tener descendientes que puedan tener instancias: una clase hoja abstracta no sirve para nada. (Pueden aparecer clases abstractas como hojas en un marco de trabajo, pero al final deben especializarse.)

Una clase concreta no debe tener operaciones abstractas, pues de lo contrario sería necesariamente abstracta, pero una clase abstracta sí puede tener operaciones concretas. Las operaciones concretas son aquellas que pueden implementarse una vez y utilizarse tal cual en toda la jerarquía de subclases. En su implementación, las operaciones concretas sólo pueden utilizar características (atributos y operaciones) conocidas por la clase en la que se declaran. Uno de los propósitos de la herencia es situar estas operaciones en superclases abstractas para que puedan ser compartidas por todas las subclases. Una operación concreta puede ser polimórfica, es decir, puede ser sobrescrita por un método de una clase descendiente, pero no es necesario que lo sea (puede ser una operación hoja). Una clase cuyas operaciones están todas implementadas puede ser abstracta, pero debe ser declarada explícitamente como tal. Una clase con una o más operaciones no implementadas es automáticamente abstracta.

La misma semántica se aplica a los casos de uso. Un caso de uso abstracto define un fragmento de comportamiento que no puede aparecer por sí mismo, pero que puede aparecer en la definición de casos de uso concretos a través de relaciones de generalización, extensión o inclusión. Llevando el comportamiento común a un caso de uso abstracto, el modelo se reduce y es más fácil de comprender.

Existe una relación similar en otros clasificadores y elementos generalizables.

Notación

El nombre de una clase u operación abstracta debe aparecer en cursiva. También se puede utilizar la palabra **abstract** en la lista de propiedades que aparece después o debajo del nombre, por ejemplo, **Cuenta {abstract}**.

Véase también nombre de clase.

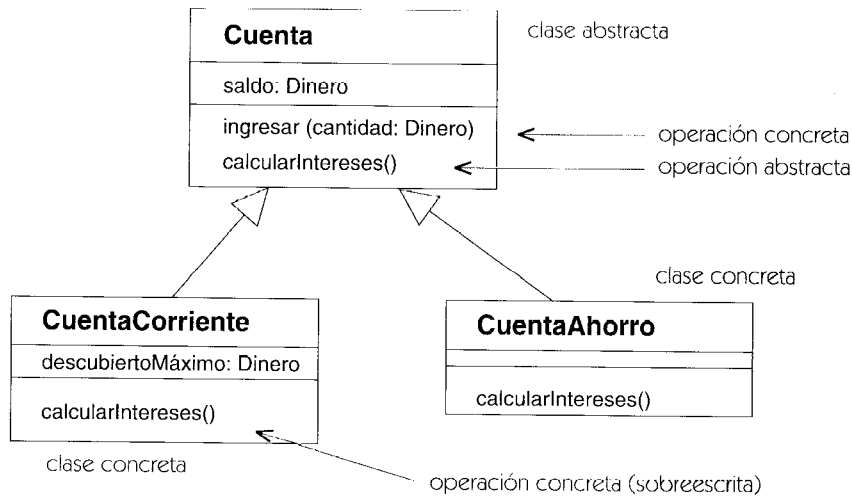


Figura 13.1 Clases abstractas y concretas

Ejemplo

La Figura 13.1 muestra una clase abstracta **Cuenta** con una operación abstracta —**calcularIntereses**— y una operación concreta —**ingresar**—, de la que se han declarado dos subclases concretas. Debido a que las subclases son concretas, cada una de ellas puede implementar la operación **calcularIntereses**. Los atributos son siempre concretos.

Discusión

La distinción entre modelar una clase como abstracta o como concreta no es tan importante como a primera vista podría parecer, pues es más una decisión de diseño sobre un modelo que una propiedad inherente al mismo. El estado de una clase puede cambiar con la evolución de un diseño. Una clase concreta puede modelarse como abstracta si se añaden subclases que incluyan todas sus posibilidades. Una clase abstracta puede ser modelada como concreta si nos encontramos con que las diferencias entre sus subclases son innecesarias y por tanto deben ser eliminadas, o si dichas diferencias pueden representarse utilizando diferentes valores de un atributo en lugar de subclases distintas.

Una forma de simplificar la decisión es adoptar el principio de diseño que afirma que todas las clases que no son hojas deben ser abstractas (y todas las clases hoja deben ser necesariamente concretas, excepto aquellas clases hoja pensadas para generalizaciones futuras). Esta regla no pertenece a UML: es un estilo que se puede o no adoptar. La razón de ser de esta “regla de la superclase abstracta” son las diferencias entre un método heredable de una superclase y un método de una clase concreta, cuyas necesidades no siempre están bien cubiertas con un único método. El método de la superclase debe hacer dos cosas: definir el caso concreto que deberán observar todos sus descendientes, e implementar el caso concreto para la clase específica en la que aparece. Estos objetivos generalmente entran en conflicto. Por otra parte, cualquier superclase no abstracta puede separarse en una superclase abstracta y una subclase hoja concreta. La superclase abstracta contendrá todos los métodos que deben ser heredados por

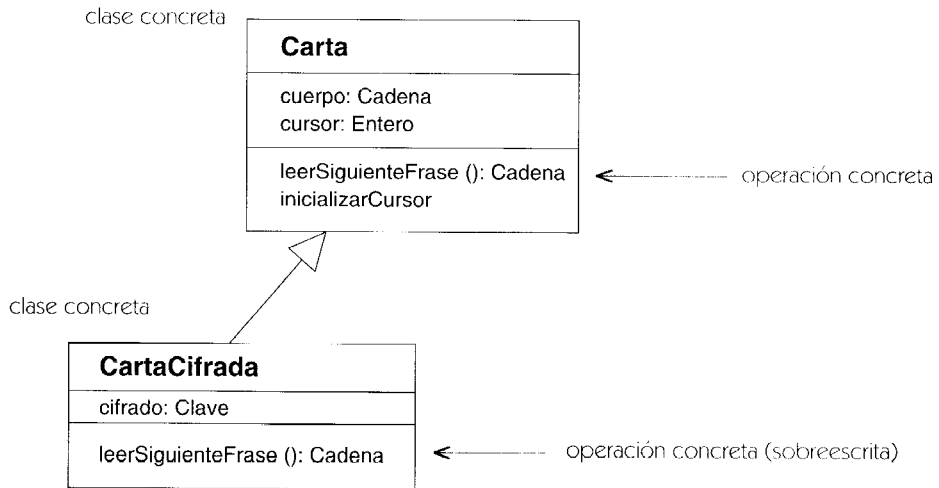


Figura 13.2 Una superclase concreta crea ambigüedad

las subclases, mientras que la subclase concreta contendrá los métodos necesarios para la clase instanciable específica. La regla de la superclase abstracta proporciona también una clara distinción entre una variable o parámetro que debe mantener el tipo concreto específico y una que puede tener el tipo de cualquiera de los descendientes de la superclase.

Considérese la declaración de la clase **Carta** de la Figura 13.2, la cual no sigue la regla de la superclase abstracta. Dicha clase tiene dos operaciones: **inicializarCursor** que pone el cursor al principio, y **leerSiguienteFrase** que devuelve el texto de la siguiente frase no leída. Sin embargo, la subclase **CartaCifrada** representa una carta cuyo texto ha sido cifrado y en la que la operación **leerSiguienteFrase** ha sido sobrescrita debido a que el texto debe ser descifrado antes de poder ser devuelto. La implementación de la operación es completamente diferente. El hecho de que **Carta** sea una superclase concreta no permite distinguir entre un parámetro que fuera una **Carta** ordinaria (no sobrescribible) y uno que pudiera ser tanto una **Carta** como una **CartaCifrada**.

El enfoque de la superclase abstracta permite distinguir la clase abstracta **Carta** (que puede ser tanto una carta cifrada como una no cifrada) y añadir la clase **CartaNoCifrada** para representar cada caso concreto, como se muestra en la Figura 13.3. En este caso, **leerSiguienteFrase** es una operación abstracta implementada por cada una de las subclases, mientras que **inicializarCursor** es una operación concreta, igual para las dos subclases. El modelo es simétrico.

Si se sigue la regla de la superclase abstracta, la declaración de clases abstractas puede determinarse automáticamente a partir de la jerarquía de clases, por lo que representarlo en diagramas es redundante.

Hay una excepción a la afirmación de que una clase hoja abstracta no sirve para nada: una clase abstracta declarada como espacio de nombres común de un conjunto de atributos y operaciones de ámbito global a la clase. Esta utilización es muy poco frecuente pero a veces es necesaria cuando se programa con lenguajes no orientados a objetos. Eso sí, debe evitarse siempre que sea posible, ya que las variables globales vulneran el espíritu del diseño orientado a objetos introduciendo dependencias globales. Una clase unitaria a menudo proporciona la misma funcionalidad de una manera más fácil de ampliar.

Véase [Gamma-95], Patrón Unitario (Singleton).

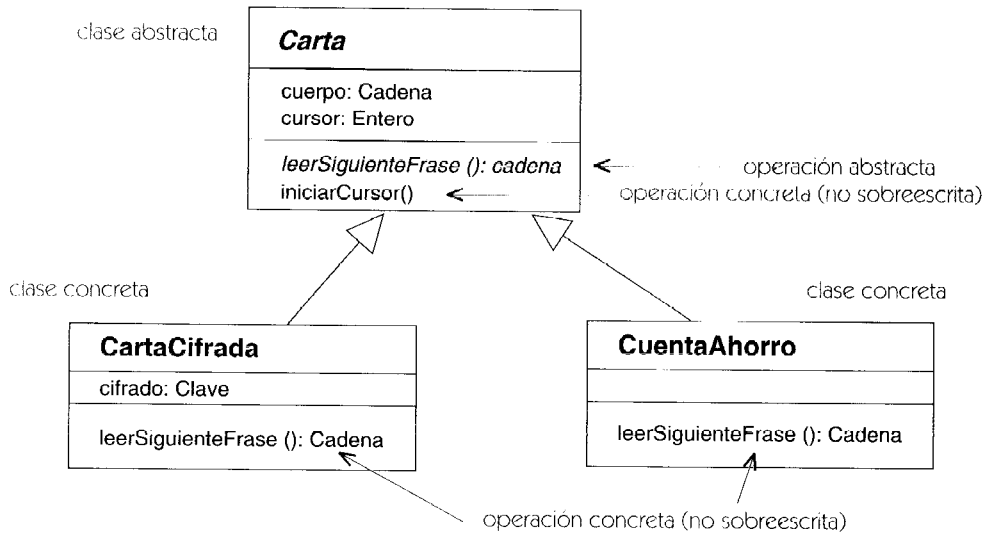


Figura 13.3 Una superclase abstracta elimina la ambigüedad

acceso

Dependencia de permiso que posibilita a un paquete hacer referencia a elementos de otro paquete.

Véase amigo/a, importar, visibilidad.

Semántica

Un paquete (cliente) que hace referencia a un elemento de otro paquete (proveedor) debe importar el paquete que contiene el elemento utilizando una dependencia «**access**» o «**import**» del paquete cliente hacia el paquete proveedor. Un paquete consigue implícitamente acceso a todos los paquetes importados por un paquete dentro del cual está anidado (esto es, los paquetes anidados pueden ver todo aquello que ven sus paquetes contenedores).

Un elemento de un paquete tiene acceso a todos los elementos que son *visibles dentro del paquete*. Las reglas de visibilidad se pueden resumir de la siguiente forma:

- Todo elemento definido en un paquete es visible dentro del mismo paquete.
- Si un elemento es visible dentro de un paquete, es visible en todos los paquetes anidados dentro del mismo.
- Si un paquete accede o importa otro paquete, todos los elementos definidos con visibilidad pública en el paquete accedido o importado son visibles dentro del paquete que los importa.
- Si un paquete es hijo de otro, los elementos definidos con visibilidad pública o privada en el paquete padre son visibles dentro del paquete hijo.

- Las dependencias de acceso e importación no son transitivas. Si A ve a B y B ve a C, no necesariamente A verá a C.

Una consecuencia de esto es que un paquete no puede ver el interior de sus propios paquetes anidados a menos que acceda a ellos y que los contenidos que haya dentro de dichos paquetes sean públicos. Algunas reglas más sobre visibilidad:

- Los contenidos de un clasificador, es decir, atributos, operaciones y clases anidadas, son visibles dentro del paquete si tienen visibilidad pública en el clasificador. Obsérvese que los contenidos no estructurados de un subsistema están gobernados por las reglas sobre visibilidad de los paquetes anteriormente establecidas, mientras que los atributos y operaciones se gobiernan por esta regla.
- Los contenidos de un clasificador son visibles dentro de un clasificador descendiente si tienen visibilidad pública o protegida en el clasificador.
- Todo lo que contiene un clasificador es visible para los elementos internos del clasificador, incluyendo métodos internos o máquinas de estados del clasificador.

El caso normal sencillo concierne a los elementos de paquetes que están al mismo nivel. En este caso, un elemento puede ver todos los elementos de su propio paquete y todos los elementos que tengan visibilidad pública de aquellos paquetes importados por su paquete. Una clase puede ver las características públicas de las clases que ve y también las características protegidas de sus ascendientes.

Notación

Una dependencia de acceso se representa mediante una flecha con línea discontinua que sale del paquete cliente y apunta al paquete proveedor. La flecha está etiquetada con la palabra clave **«access»**.

Discusión

La Figura 13.4 muestra un ejemplo de acceso entre dos paquetes que se encuentran al mismo nivel. El paquete P puede acceder al paquete Q, pero el paquete Q no puede acceder a P. Las clases K y L del paquete P pueden ver la clase pública M del paquete Q, pero no la clase privada N. Las clases M y N no pueden ver ninguna clase del paquete P, aunque la visibilidad de la clase K es pública, debido a que Q no tiene acceso a P. Para que una clase de un paquete Q sea visible en un paquete P que está al mismo nivel que Q, la clase debe tener visibilidad pública, y Q debe ser accedido o importado por el paquete P.

La Figura 13.5 muestra un caso de visibilidad y declaraciones de acceso algo más complejo. El símbolo que precede al nombre del elemento representa la visibilidad del mismo fuera de su propio contenedor: + para visibilidad pública, # para protegida (visible sólo en descendientes) y – para visibilidad privada (no es visible fuera del contenedor).

La clase A puede ver las clases C y E porque está dentro de los paquetes X e Y.

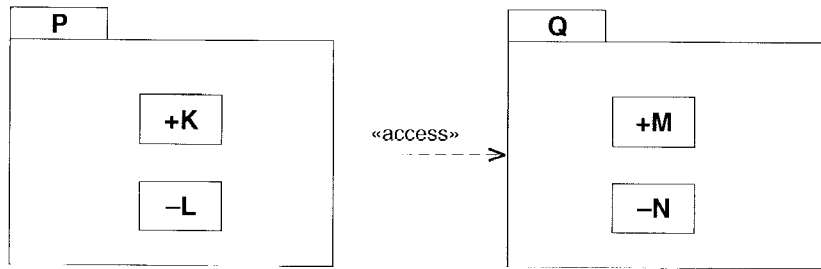


Figura 13.4 Acceso entre iguales

Las clases C y A pueden ver la clase D porque el paquete Y importa el paquete Z. La clase A está anidada dentro del paquete Y, por lo que puede ver todo aquello que ve Y.

Las clases A, C y E pueden ver a B porque están anidadas en el paquete X, que implementa el paquete V el cual contiene a B. Sin embargo, no pueden ver a F porque tiene visibilidad privada dentro de su propio paquete V. La clase F no es visible fuera de V.

La clase E no ve a D porque D está en el paquete Z, que no ha sido importado por el paquete X.

Ni la clase C ni la clase E pueden ver la clase A, debido a que A está en el paquete U, que no ha sido importado por ningún otro paquete.

Las clases B y F ven las clases D y E que están en paquetes que contienen al paquete V, al que ellas pertenecen. También pueden ver la clase C, que está en el paquete Y, pues dicho paquete es importado por el paquete X que engloba a V. El hecho de que F sea privada no limita el hecho de que pueda ver otras clases, pero evita que otras clases la vean a ella.

Las clases B y F se ven mutuamente porque están en el mismo paquete. La clase F es privada para las clases de otros paquetes, pero no para las que están en su mismo paquete.

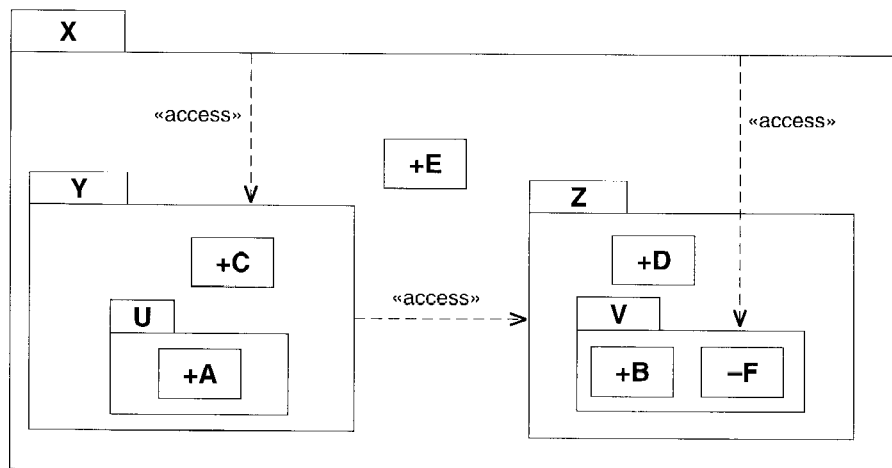


Figura 13.5 Reglas de acceso

acción

Computación atómica ejecutable que produce un cambio en el estado del modelo o que devuelve un valor. Contrastar con: actividad.

Véase también acción de entrada, acción de salida, transición.

Semántica

Una acción es una computación atómica, es decir, no puede ser finalizada por una orden de terminación externa. Puede asociarse a una transición en una máquina de estados (entre dos estados o bien dentro de un estado simple) o a un paso en una interacción. Generalmente es una operación primitiva o pseudo-primitiva sobre el estado de un sistema, a menudo sobre el estado de un objeto simple. Las acciones típicas son: asignación de un valor a un atributo, acceso a la información de atributos o enlaces, creación de nuevos objetos o enlaces, operaciones aritméticas sencillas y envío de señales a otros objetos. Las acciones son los pasos discretos con que se construye el comportamiento. Se supone que las acciones tienen que ser computaciones “rápidas” para no perjudicar el tiempo de respuesta del sistema. El sistema puede ejecutar varias acciones simultáneamente y hacer que compartan el tiempo entre ellas, pero su ejecución debe ser independiente.

Las acciones pueden estar asociadas a una transición, ejecutándose cuando la transición se dispara. También pueden aparecer como acciones de entrada o de salida de los estados. Hay acciones disparadas por transiciones que entran o dejan un estado. Todas las acciones son atómicas, esto es, se ejecutan completamente y sin interrupción de otras operaciones.

Una actividad es también una computación, pero puede tener una estructura interna y recibir una orden de terminación externa de una transición de un evento externo, y puede asociarse a un estado pero no a una transición. A diferencia de las acciones, las actividades pueden persistir indefinidamente hasta ser finalizadas externamente, aunque también pueden terminar por sí mismas. Una acción no puede ser finalizada externamente y debe estar asociada a una transición o a la entrada o salida de un estado, nunca a un estado en sí mismo.

Estructura

Una acción tiene un conjunto de objetos destino, una referencia a la señal a enviar o a la operación a realizar (a los que colectivamente se conoce como *solicitud*), una lista de valores para los argumentos y una expresión opcional de recurrencia que especifica la posible iteración.

Conjunto de objetos destino. Expresión de conjunto de objetos destino que produce un conjunto de objetos. En muchos casos, el conjunto contiene un objeto simple fijo. A todos los objetos del conjunto se les envía concurrentemente una copia del mensaje con la lista de argumentos correspondiente (es decir, se les comunica mediante difusión). Cada uno de ellos recibe y manipula independientemente una instancia distinta del mensaje. Si el conjunto está vacío no ocurre nada.

Solicitud. Denota la declaración de una operación o una señal. O bien se envía la señal a los objetos o bien se llama a la operación (en el caso de que la operación devuelva un valor, el conjunto debe contener un único objeto).

Lista de argumentos. Una lista de argumentos. Cuando se evalúa, los valores de la lista de argumentos deben corresponderse con los parámetros de la señal o de la operación. Los argumentos se proporcionan como parte de la llamada o del envío de la señal.

Recurrencia. Expresión de una iteración, que especifica cuántas veces hay que realizar la acción, y que opcionalmente puede incluir variables de iteración. Esta expresión también puede describir una acción condicional (una iteración con una o ninguna repetición).

Tipos de acciones

Acción de asignación. Una asignación es una acción que da a un atributo de un objeto un valor determinado. La acción tiene una expresión para el objeto destino (el nombre de un atributo del objeto), y una expresión para el valor a asignar al atributo del objeto.

Acción de llamada. Una acción de llamada provoca una invocación de una operación sobre un objeto: es una invocación a una operación de un objeto. La acción tiene un nombre de mensaje, una expresión para la lista de argumentos, y una expresión de conjunto de objetos destino. El destino puede ser un conjunto de objetos, en cuyo caso la llamada se produce concurrentemente y la operación no deberá por tanto devolver ningún valor. Si la operación devuelve un valor, el destino debe ser un único objeto.

Las acciones de llamada son asíncronas. El que realiza la llamada debe esperar a que se complete la operación invocada antes de recibir de nuevo el control. Si la operación se implementa como un evento de llamada, el que realiza la llamada espera hasta que el receptor ejecute la transición disparada por la llamada para retomar el control. Si la ejecución de la operación devuelve valores, el que realiza la llamada los recibe cuando retoma el control.

Acción de creación. Una acción de creación produce la instanciación y creación de un objeto (véase creación). La acción tiene una referencia a una clase, y opcionalmente una operación presente en la clase con una lista de argumentos. La ejecución de la acción crea una nueva instancia de la clase, cuyos atributos obtienen los valores correspondientes a la evaluación de las expresiones de valor iniciales. Si existe una operación de creación explícita, se ejecuta. La operación puede sobrescribir la inicialización de los valores de los atributos, normalmente utilizando valores como argumentos de la acción de creación.

Acción de destrucción. Una acción de destrucción produce la destrucción de un objeto destino. No existen otros argumentos. El resultado de ejecutar la acción es la destrucción del objeto, junto con todos sus enlaces y con todas las partes de las que estuviera compuesto (véase composición).

Acción de retorno. Una acción de retorno provoca la transferencia del flujo de objeto a aquél que llamó a la operación. Esta acción sólo está permitida dentro de una operación invocada por una llamada. La acción tiene una lista opcional de valores devueltos que se pone a disposición de quien realizó la llamada cuando se le devuelve el control. Si la acción adjunta fue invocada de forma asíncrona, el que llamó a la operación debe escoger explícitamente entre recibir el mensaje de retorno (como una señal), o perderlo.

Acción de envío. Una acción de envío crea una instancia de una señal y la inicializa con los argumentos obtenidos al evaluar las expresiones de argumento de la acción. La señal se envía a los objetos del conjunto de objetos destino obtenido evaluando la expresión destino de la

acción. Cada objeto del conjunto recibe su propia copia de la señal. El que envía la señal tiene su propio hilo de control y continúa ejecutándose, ya que el envío de una señal es asíncrono. La acción tiene un nombre de señal, una lista de expresiones para los argumentos de la señal y una expresión de conjunto de objetos destino para los objetos destino.

Si se omite el conjunto de objetos destino, la señal se envía a uno o más objetos determinados por la señal y por la configuración del sistema. Por ejemplo, una excepción se envía a un ámbito cerrado que viene determinado por las políticas del sistema.

Acción de terminación. Una acción de terminación provoca la destrucción del objeto propietario de la máquina de estados que contiene la acción (esto es, se suicida). La destrucción de un objeto es un evento al que pueden responder otros objetos.

Acción no interpretada. Una acción no interpretada es una estructura de control u otra estructura no definida en UML.

Notación

UML no tiene un lenguaje fijo de acciones. Se espera que los que realizan los modelos elegirán un lenguaje de programación real para escribir las acciones. La siguiente adaptación de OCL se utiliza en este libro para escribir pseudocódigo, pero no es parte del estándar.

Acción de asignación

destino := expresión

Acción de llamada

conjunto-objetos-destino . nombre-operación (argumento_{lista})

Acción de creación

new nombre-clase (argumento_{lista})

Acción de destrucción

objeto . destroy ()

Acción de retorno

return expresión_{lista}

Acción de envío

conjunto-objetos-destino . nombre-señal (argumento_{lista})

Acción de terminación

terminate

Acción no interpretada

if (expresión) **then** (acción) **else** (acción)

Si es necesario distinguir explícitamente entre llamada y envío, se pueden anteponer las palabras clave **call** o **send** a la expresión de forma opcional.

Discusión

La especificación de UML define un conjunto de acciones con la esperanza de que se añadirán otras en la implementación real de las herramientas que trabajen con UML. Esta decisión fue el resultado del compromiso entre el deseo de precisión y la necesidad de que los desarrolladores trabajaran con varios lenguajes, lo cual abarca un amplio espectro de conceptos semánticos. Existen muchas más variaciones en la semántica de ejecución entre lenguajes de programación de la que existe en las estructuras de datos o en el conjunto de estructuras de control disponibles. Las diferencias sutiles entre lenguajes son difíciles de plasmar de forma práctica, sin tener en cuenta si es posible hacerlo teóricamente. La selección de un lenguaje de programación como base para un lenguaje de acciones podría, sin embargo, tener el efecto de desanimar a los que utilizan otros lenguajes, lo cual no queríamos hacer. La semántica de acciones ha quedado pues algo incompleta y ambigua dentro del propio UML. Para precisar la semántica, hay que añadir a UML la semántica del lenguaje de acciones (a menudo un lenguaje estándar de programación) que se está utilizando. Algunos críticos se han quejado de que UML es impreciso debido a su libertad, pero lo es sólo hasta el grado de imprecisión del lenguaje de acción elegido. El auténtico defecto es que UML no impone una *lingua franca* de acciones y otras expresiones, lo cual es posible sólo a duras penas en el políglota mundo actual de la informática a pesar de las emotivas peticiones al respecto.

acción asíncrona

Solicitud en la cual el objeto emisor no hace una pausa para esperar los resultados. Envío.

Véase enviar, acción síncrona.

acción de entrada

Una acción realizada cuando se entra en un estado.

Véase también acción de salida, atómico/a, máquina de estados, transición.

Semántica

Un estado puede tener una acción de entrada opcional unida a él. Siempre que se entra en el estado se ejecuta la acción de entrada, después de las acciones asociadas a las transiciones entrantes o a la salida de los estados previos y antes de cualquier acción asociada a estados internos. La acción de entrada no se puede evadir por ningún medio. Está garantizado que se ha ejecutado siempre que el estado que la posee o un estado anidado dentro de él esté activo.

Orden de ejecución. En una transición entre dos estados con acciones de entrada y salida en las cuales la transición también tiene una acción, el orden de la ejecución es: Cualquier acción

de salida se ejecuta en el estado origen y los estados a los que engloba hacia fuera, pero no incluye a los estados que engloban tanto al estado de origen como al destino. Entonces se ejecuta la acción en la transición, después de lo cual se ejecutan las acciones de la entrada (las más externas primero) en los estados englobados dentro del estado, hacia abajo incluyendo el estado destino. La Figura 13.96 muestra algunas transiciones con acciones múltiples.

Notación

Una acción de entrada se codifica usando la sintaxis para una transición interna con el nombre de la **entrada** del evento simulado (que es por lo tanto una palabra reservada y no se puede utilizar como nombre real del evento).

`entrada / secuencia-acción o entry / secuencia-acción`

Sólo se puede unir una acción de entrada a un estado, pero la acción puede ser una secuencia, de modo que no se pierde generalidad.

Discusión

Las acciones de entrada y salida no son esenciales semánticamente (la acción de entrada se podría unir a todas las transiciones entrantes) pero facilitan la encapsulación de un estado para poder separar el uso externo de él de su construcción interna. Permiten definir acciones de inicialización y de finalización, sin la preocupación de que puedan evitarse. Son particularmente útiles con excepciones, porque definen las acciones que deben ser realizadas incluso si ocurre una excepción.

Una acción de entrada es útil para realizar una inicialización que deba hacerse cuando se entra al estado por primera vez. Un uso es inicializar las variables que capturan información durante un estado.

Por ejemplo, una interfaz de usuario para permitir la entrada, por un teclado numérico, de un número de teléfono o de un número de cuenta, limpiaría el número de la entrada. El reajuste de un contador de error, tal como el número de fallos en la contraseña, es otro ejemplo. La asignación de almacenamiento temporal, necesario durante el estado, es otro uso para una acción de entrada.

A menudo se utilizan juntas una acción de entrada y una acción de salida. La acción de entrada asigna recursos, y la acción de salida los libera. Incluso si ocurre una transición externa, se liberan los recursos. Esto es una buena manera de manejar errores de usuario y excepciones. Los errores de usuario accionan transiciones de alto nivel que abortan estados anidados, pero los estados anidados tienen una oportunidad de limpiar antes de perder el flujo de control.

acción de salida

Una acción que se realiza cuando se sale de un estado.

Véase también acción de entrada, atómico/a, máquina de estados, transición.

Semántica

Un estado puede tener asociada una acción de salida. Se ejecuta siempre que se salga del estado de cualquier manera, después de cualquier otra acción asociada a estados o transiciones más internos y antes de cualquier acción unida a estados más externos. La acción de salida no se puede evadir por ningún medio. Está garantizado que se ejecutará antes de que el flujo de objeto abandone el estado que la posee.

Las acciones de entrada y de salida no son esenciales semánticamente (la acción de salida se podría unir a todas las transiciones salientes), pero facilitan la encapsulación de un estado para poder separar el uso externo de él de su construcción interna. Permiten definir acciones de inicialización y de terminación, sin la preocupación de que puedan ser evitadas. Son particularmente útiles con excepciones, porque definen las acciones a realizar incluso si ocurre una excepción.

Notación

Una acción de salida se codifica usando la sintaxis para una transición interna con el evento simulado llamado **salida** (que es, por lo tanto, una palabra reservada y no se puede utilizar como nombre real del evento).

`salida / secuencia-acción o exit / secuencia-acción`

A un estado solamente se puede asociar una acción de salida, pero la acción puede ser una secuencia de acciones, así que no se pierde ninguna generalidad.

Discusión

Una acción de salida es útil para realizar una limpieza general cuando se sale de un estado. El uso más significativo de las acciones de salida es liberar el almacenamiento temporal y otros recursos asignados durante la ejecución del estado (generalmente, un estado con el detalle anidado).

A menudo se utilizan juntas una acción de entrada y una acción de salida. La acción de entrada asigna recursos, y la acción de salida los libera. Los recursos son liberados incluso si ocurre una excepción.

acción síncrona

Solicitud en la cual el objeto emisor se detiene a esperar una respuesta; también es una llamada. Contraste con: acción asíncrona.

activación

Ejecución de una operación. Una activación (también conocida como foco de control) representa el período durante el cual un objeto realiza una operación, bien directamente o bien a

través de una operación subordinada. Modela tanto la duración de la ejecución como la relación de control entre la ejecución y los que hicieron la llamada a la misma. En un computador y lenguaje de programación convencionales, una activación se correspondería con un bloque de llamada en la pila.

Véase llamada, diagrama de secuencia.

Semántica

Una activación es una instancia de ejecución de una operación, que incluye el período durante el cual la operación llama a otras operaciones subordinadas (véase llamada). Su contexto comprende un conjunto de variables locales accesibles únicamente por la activación, una posición actual dentro del método (u otra descripción de comportamiento), y una referencia (la *referencia de retorno*) a la activación que representa el contexto de llamada, que retoma el control cuando termina la activación actual. Una activación sin referencia de retorno debe ser el resultado de una transición sobre la máquina de estados de la clase de un objeto activo; cuando se completa la activación, la máquina de estados simplemente espera el próximo evento.

Obsérvese que esta definición describe un procedimiento ordinario, tal y como se implementa en una típica máquina de von Neumann. Pero expresarlo de forma general significa aplicarlo también a un entorno distribuido, en el cual no hay memoria compartida y en el que el marco de pila está formado por una lista enlazada de activaciones en diferentes espacios de memoria.

Notación

Una activación se representa en un diagrama de secuencia utilizando un rectángulo alto y delgado (una barra vertical hueca) cuya parte superior está alineada con el momento de su iniciación y cuya parte inferior lo está con el momento de su finalización. La operación a realizar se representa mediante una etiqueta de texto al lado del símbolo de activación o en el margen izquierdo, dependiendo del estilo.

De forma alternativa, también es posible que el símbolo de mensaje entrante indique la operación, en cuyo caso se puede omitir la etiqueta en la propia activación. Si el flujo de objeto es de procedimientos, entonces el extremo superior del símbolo de activación estará en la punta de la flecha del mensaje entrante que inicia la acción, y del extremo inferior saldrá la flecha del mensaje de retorno.

Si hay actividad concurrente entre varios objetos, la activación muestra la ejecución de un objeto concurrente. A menos que los objetos se comuniquen, las actividades concurrentes son independientes, y no es relevante reflejar sus tiempos relativos de ejecución.

Cuando se trata de código de procedimientos, una activación muestra o bien la duración durante la cual un procedimiento está activo en el objeto o bien el tiempo durante el que está activo un procedimiento subordinado llamado por el procedimiento original, posiblemente en algún otro objeto. En otras palabras, se muestran simultáneamente todas las activaciones de procedimientos anidados. Este conjunto de activaciones anidadas simultáneas forma el marco de pila de la computación en un computador convencional. En caso de una segunda llamada a un objeto con una activación existente, el segundo símbolo de activación se dibuja ligeramente a la derecha del primero, de forma que den la sensación de estar apilados. Las llamadas

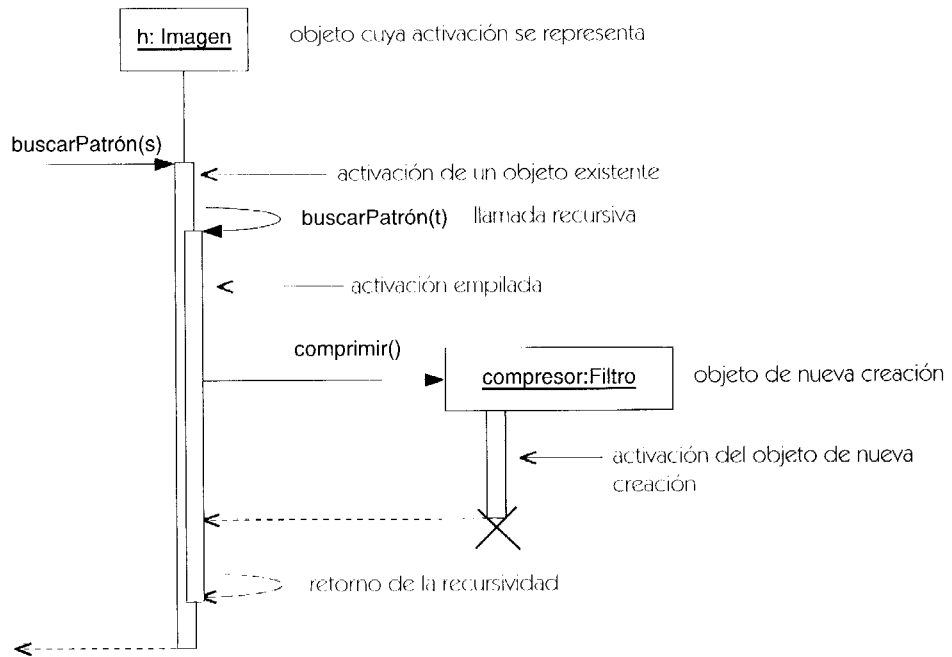


Figura 13.6 Activaciones

apiladas pueden tener cualquier nivel de anidamiento. Las llamadas pueden ser a la misma operación (llamada recursiva) o a diferentes operaciones del mismo objeto.

Ejemplo

La Figura 13.6 muestra las activaciones producidas a partir de las correspondientes llamadas, incluyendo una llamada recursiva.

actividad

Ejecución no atómica en curso dentro de una máquina de estados. Contrastar con: acción.

Véase también transición de finalización, estado.

Semántica

Una actividad es la ejecución de una subestructura dentro de una máquina de estados, pero de una subestructura que tiene una duración y posibles puntos de interrupción. Cualquier transición que fuerce la salida de la región de control aborta la actividad. Una actividad no termina por el hecho de que se dispare una transición interna, porque no hay cambio de estado, pero la acción asociada a la transición interna sí podría terminarla explícitamente.

Una actividad se puede modelar mediante estados anidados, utilizando una referencia a una submáquina o empleando una expresión de actividad.

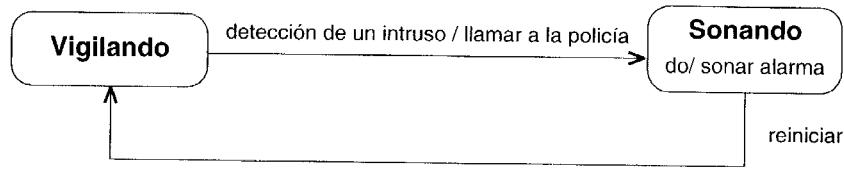


Figura 13.7 Acción y actividad

Ejemplo

La Figura 13.7 muestra un sistema de alarma que ilustra la diferencia entre una acción y una actividad. Cuando se produce el evento **detección de un intruso**, el sistema dispara una transición. Como parte de la transición, se lleva a cabo la acción **llamar a la policía**, que es una acción puesto que es algo atómico (y normalmente rápido). Mientras se ejecuta la acción no se acepta ningún evento. Después de realizar la acción el sistema pasa al estado **Sonando**, y mientras está en este estado realiza la actividad **sonar alarma**. Una actividad tarda un tiempo en realizarse, tiempo durante el cual la actividad puede ser interrumpida por algún evento. En este caso, la actividad **sonar alarma** no termina por sí sola, ya que continúa ejecutándose mientras el sistema se encuentre en el estado **Sonando**. Cuando se produce el evento **reiniciar**, se dispara la transición que devuelve al sistema al estado **Vigilando**.

Cuando el estado **Sonando** deja de estar activo, su actividad **sonar alarma** termina.

activo

Estado en el que se ha entrado y del que aún no se ha salido; estado que mantiene un objeto.

Véase también clase activa, objeto activo.

Semántica

Un estado pasa a estar activo cuando se dispara una transición de entrada, y deja de serlo cuando se dispara una transición de salida. Si un objeto tiene un hilo de control, entonces al menos hay un estado activo (en un caso extremo, una clase puede tener sólo un estado, lo que supone que la respuesta ante un evento es siempre la misma). Si un estado está activo dentro de la máquina de estados de la clase de un objeto, se dice que el objeto *mantiene* el estado, o que *está* en ese estado.

Un objeto puede tener más de un estado simultáneamente. El conjunto de estados activos se denomina configuración del estado activo. Si un estado anidado está activo, todos los estados que lo contienen están también activos. Si el objeto permite concurrencia, puede haber más de un subestado activo al mismo tiempo. Cada transición afecta, al menos, a unos pocos estados de la configuración del estado activo. Ante una transición, los estados activos no afectados por la misma permanecen activos.

Un estado compuesto puede ser secuencial o concurrente. Si es secuencial y está activo, entonces habrá exactamente uno de sus subestados activo. Si es concurrente y está activo, en-

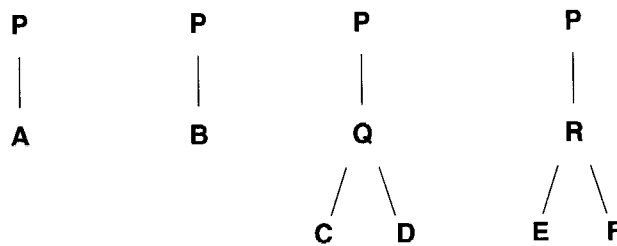
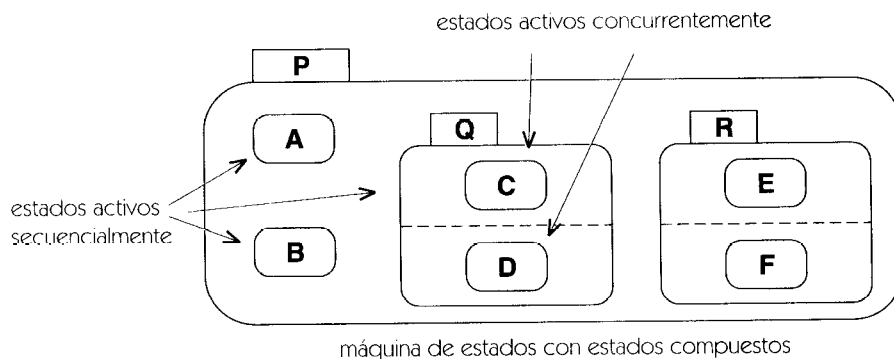
tonces todos los subestados que contiene estarán activos. En otras palabras, un estado compuesto se expande en un árbol Y-O de subestados activos, de forma que en cada nivel hay siempre estados activos.

Una transición que atraviesa en la frontera de un estado compuesto debe estructurarse de forma que mantenga las restricciones de concurrencia. Una transición en un estado secuencial compuesto normalmente tiene un estado origen y otro destino, y disparar una transición de este tipo no cambia el número de estados activos. Sin embargo, una transición en un estado concurrente compuesto tendrá un estado origen y uno destino para cada una de las regiones en que se divide el estado concurrente compuesto, y una transición de este tipo se denomina división. Si se omiten una o más regiones como destino, el estado inicial de cada región omitida está presente de forma implícita como destino, pero si una de las regiones carece de estado inicial, entonces se dice que el modelo está mal modelado. El disparar una transición de este tipo incrementa el número de estados activos, situación que se invierte a la salida del estado concurrente compuesto.

Véase máquina de estados, donde se lleva a cabo una completa discusión sobre la semántica de los estados concurrentes y las transiciones complejas.

Ejemplo

La parte superior de la Figura 13.8 muestra un ejemplo de máquina de estados que tiene estados compuestos, tanto secuenciales como concurrentes. Se han omitido las transiciones para



posibles configuraciones del estado activo

Figura 13.8 Estados activos concurrentemente

centrar la atención en los estados. La parte inferior de la figura muestra las diversas configuraciones de estados que pueden estar activas concurrentemente.

En este ejemplo hay cuatro posibles configuraciones de estados activos. Sólo los estados hoja son concretos, ya que los estados de más alto nivel son abstractos, es decir, un objeto no puede estar en uno de ellos sin estar también en uno de sus estados hoja anidados. Por ejemplo, un objeto no puede estar en el estado Q sin estar en alguno de los subestados de Q. Como Q es concurrente, tanto C como D tienen que estar activos si Q lo está. Cada estado hoja se corresponde con un hilo de control. En un ejemplo más amplio, el número de posibles configuraciones crecería exponencialmente y sería imposible mostrarlas todas, de aquí la ventaja de la notación.

actor

Abstracción de las entidades externas a un sistema, subsistemas o clases que interactúan directamente con el sistema. Un actor participa en un caso de uso o conjunto coherente de casos de uso para llevar a cabo un propósito global.

Véase también caso de uso.

Semántica

Un actor abstrae y caracteriza a un usuario externo o a un conjunto de usuarios externos relacionados que interactúan con el sistema o clasificador. Es una idealización con un propósito y significado concretos, que puede no corresponderse con objetos físicos. Un objeto físico puede combinar propósitos dispares y por tanto ser modelado por varios actores y viceversa, diferentes objetos físicos podrían incluir el mismo propósito, y por tanto ser modelados como el mismo actor. El objeto usuario puede ser una persona, un sistema informático, otro subsistema u otro tipo de objeto. Por ejemplo, el conjunto de actores de un sistema de redes de computadores podría incluir los actores **Operador**, **Administrador del Sistema**, **Administrador de Bases de Datos** y **Usuario** sin más. También podría haber actores no humanos como por ejemplo **Cliente Remoto**, **Reloj Maestro** o **Impresora de Red**.

Cada actor define un conjunto de roles que los usuarios de un sistema asumen cuando interactúan con el mismo. El conjunto completo de actores describe todas las diferentes formas de comunicación entre los usuarios externos y el sistema. Cuando se implementa el sistema, los actores se implementan como objetos físicos. Un objeto físico puede implementar más de un actor si puede llevar a cabo todos sus roles. Por ejemplo, una persona puede ser tanto vendedor como cliente de una misma tienda; estos actores no están inherentemente relacionados, pero ambos pueden ser implementados por una persona. Cuando se diseña un sistema, los diferentes actores se realizan mediante clases de diseño (véase realización).

Las diferentes interacciones de los actores con un sistema se cuantifican en casos de uso. Un caso de uso es una pieza coherente de funcionalidad que implica a un sistema y a sus actores para llevar a cabo algo que tiene sentido para los actores. Un caso de uso puede implicar a uno o más actores, de la misma forma que un mismo actor puede participar en más de un caso de uso. Los actores vienen determinados por los casos de uso que realizan y por los roles que

desempeñan en los diferentes casos de uso. Un actor que no participa en ningún caso de uso carece de sentido.

Un modelo de casos de uso caracteriza los tipos de comportamiento que proporciona una entidad, como por ejemplo un sistema, un subsistema o una clase, en sus interacciones con entidades externas. En el caso de un sistema, los actores pueden ser realizados tanto por usuarios humanos como por otros subsistemas. Cuando se trata de un subsistema o de una clase, los elementos externos pueden ser actores del sistema global o pueden ser otros elementos internos del sistema, como otros subsistemas u otras clases.

Las instancias de actores se comunican con el sistema mediante el envío y recepción de instancias de mensajes (señales y llamadas) desde y hacia instancias de casos de uso, y en el nivel de realización, desde y hacia los objetos que implementan el caso de uso. Todo lo anterior se expresa mediante asociaciones entre el actor y el caso de uso.

Un actor puede listar el conjunto de señales que envía y que recibe, y mantener una lista de las interfaces que ofrece y que requiere. Las interfaces de un actor deben ser compatibles con las de los casos de uso con los que se comunica. En otras palabras, un actor debe recibir todas las señales que un caso de uso sea capaz de enviarle y no debe mandarle al caso de uso señales que éste no sea capaz de recibir. Las interfaces de un actor restringen las clases con las que pueda corresponderse (que puedan implementar su comportamiento). Un actor puede también tener una lista de atributos que caracterice su estado.

Generalización

Pueden existir semejanzas entre dos o más actores; esto es, pueden comunicarse con el mismo conjunto de casos de uso de la misma manera. Esta semejanza se expresa mediante la generalización en otro actor, posiblemente abstracto, que modele los aspectos comunes. Los actores descendientes heredarán los roles y relaciones con casos de uso del actor antecesor. Una instancia de un actor descendiente siempre se puede utilizar en aquellos casos en los que se espera una instancia del actor antecesor (principio de capacidad de sustitución). Un descendiente incluye los atributos y operaciones de sus antecesores.

Notación

Un actor se puede representar mediante un símbolo de clase (rectángulo) con el estereotipo «actor». El icono estándar de un actor es el “monigote”, con el nombre del actor debajo. El actor puede tener compartimentos que muestren los atributos y eventos que recibe, y también dependencias que muestren los eventos que envía, como cualquier clasificador normal (Figura 13.9).



Figura 13.9 Símbolo de actor

agregación

Forma de asociación que especifica una relación todo-parte entre un agregado (el todo) y las partes que lo componen.

Véase también composición.

Semántica

Una *asociación binaria* puede declararse como agregación, o lo que es lo mismo, como relación todo-parte. Un extremo de la asociación se designa como agregado mientras que el otro no se marca de forma especial. No pueden ser agregados (o compuestos) ambos extremos, pero sí que pueden estar los dos sin marcar, en cuyo caso no se trataría de una agregación.

Los enlaces instanciados de las asociaciones de agregación obedecen ciertas reglas. La relación de agregación es transitiva y antisimétrica a través de los enlaces de la agregación, incluso a través de aquellos que pertenecen a diferentes relaciones de agregación. La propiedad transitiva significa que tiene sentido decir que “B es parte de A” si existe una ruta de enlaces de agregación desde B hasta A en sentido directo (en este ejemplo, de las partes hacia el todo). La antisimetría obliga a que no existan bucles en las rutas dirigidas de los enlaces de agregación. Esto es, no es posible que ningún objeto sea parte de sí mismo directa o indirectamente. Uniendo los dos roles, el grafo de enlaces de agregación de todas las asociaciones de agregación es un grafo parcialmente ordenado, un grafo sin bucles (el caso más común de orden parcial sería un árbol). En la Figura 13.10 se muestra un ejemplo.

Una ruta dirigida de enlaces desde el objeto B hasta el objeto A implica que existe una ruta dirigida de asociaciones de agregación de la clase B a la clase A, pero la ruta de asociaciones puede implicar bucles en los que la misma clase aparezca más de una vez. Una ruta dirigida de asociaciones de agregación de una clase a sí misma es una *recursividad*.

Existe una relación más fuerte de agregación llamada composición. Un compuesto es un agregado con las restricciones adicionales de que un objeto puede ser parte sólo de un compuesto, y de que el objeto compuesto tiene la responsabilidad de la disponibilidad de sus partes a la hora de crearlas y destruirlas.

Véase composición para más detalles.

En la agregación simple, una parte puede pertenecer a más de un agregado, y lo que es más, puede existir independientemente del agregado. A menudo el agregado “necesita” las partes, en el sentido de que puede ser considerado como una colección de partes, pero las partes pueden existir por sí mismas, sin tener que ser vistas como partes únicamente. Por ejemplo, una ruta es poco más que un conjunto de segmentos, pero un segmento puede existir independientemente de si pertenece o no a una ruta, y además el mismo segmento puede pertenecer a más de una ruta.

Véase asociación y extremo de asociación para más información sobre las propiedades de la agregación.

Notación

Una agregación se representa mediante un rombo o diamante hueco en el extremo de la línea de la asociación que se conecta a la clase agregada (véase Figura 13.11). Si la agregación es una composición, su representación es un rombo relleno (véase Figura 13.54). No pueden tener indicadores de agregación ambos extremos.

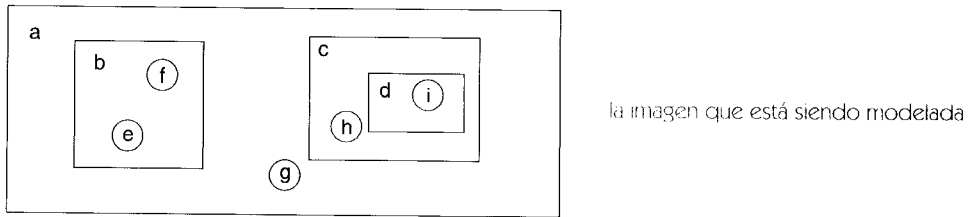


diagrama de clases: recursividad empleando agregación

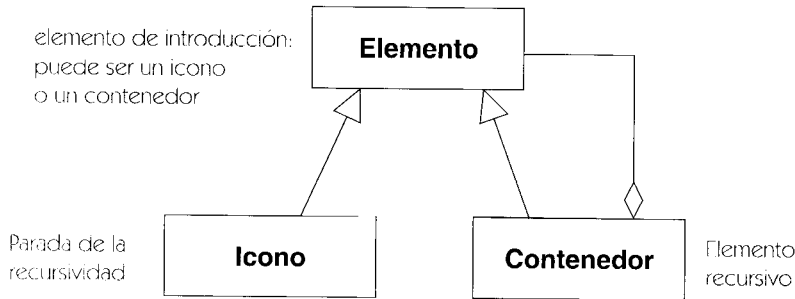


diagrama de objetos: no hay bucles entre los objetos

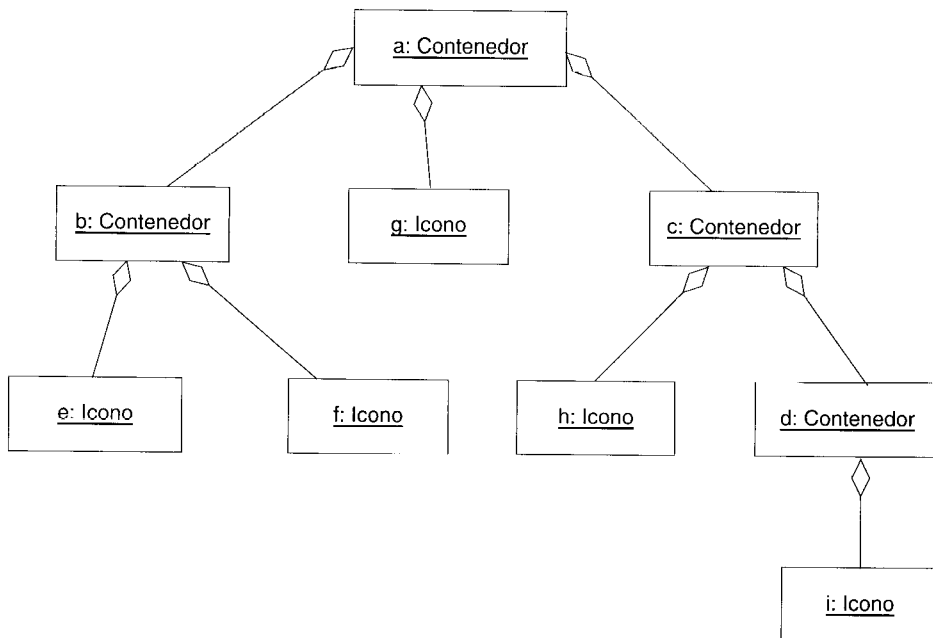


Figura 13.10 Las agregaciones de objetos son acíclicas

Una clase agregada tiene múltiples partes, pero cada relación entre la clase agregada y cada una de las partes es una asociación distinta (véase Figura 13.12).

Si hay dos o más asociaciones de agregación para la misma clase agregada, se pueden representar como un árbol combinando los extremos de la agregación en un único segmento

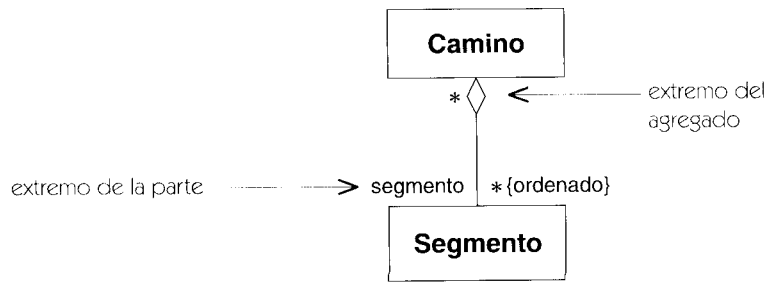


Figura 13.11 Notación de agregación

(véase Figura 13.13), pero esto requiere que todos los adornos de los extremos de la agregación sean consistentes; por ejemplo, deben tener la misma multiplicidad. Dibujarlo mediante un árbol es una mera cuestión de presentación que no aporta ninguna semántica adicional.

Discusión

La distinción entre asociación y agregación es muy a menudo un asunto de gustos más que una diferencia semántica real. Hay que recordar que la agregación es una asociación. La agregación conlleva la idea de que el agregado es la suma de sus partes, y en efecto, la única semántica adicional que aporta a la asociación es la restricción que impide que en los enlaces de agregación existan bucles, que es importante conocer. Otras restricciones, como la existencia de dependencia, las expresa la multiplicidad, no la agregación. A pesar del poco significado que

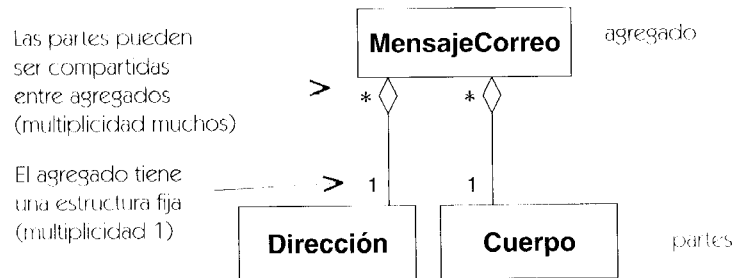


Figura 13.12 Un agregado con varias partes

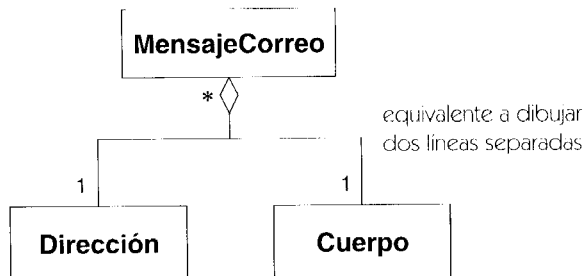


Figura 13.13 Representación en árbol para múltiples agregaciones de la misma clase

aporta, todo el mundo está convencido de que es necesaria (por diferentes razones). Piense en ello como en un placebo de modelado.

Hay algunas propiedades secundarias conectadas con la agregación, pero no son suficientes para requerir formar parte de la definición. Algunas de ellas son la propagación de operaciones desde el agregado a las partes (por ejemplo una operación mover), o la asignación de memoria compacta (de forma que el agregado y sus partes puedan cargarse eficientemente con una única transferencia de memoria). Algunos autores distinguen varios tipos de agregación, pero dichas distinciones son tan sutiles que probablemente sean innecesarias para un modelado común.

La agregación es una propiedad que trasciende una asociación particular. Se pueden componer agregaciones sobre diferentes pares de clases, y el resultado es una agregación. La agregación impone la restricción sobre las instancias de todas las asociaciones de agregación (incluyendo las asociaciones de composición) de que no pueden existir bucles entre los enlaces de agregación, incluyendo los enlaces de asociaciones diferentes. En cierto sentido, la agregación es una especie de generalización de la asociación en la cual las restricciones y algunas operaciones se aplican a asociaciones de muchos tipos específicos.

La composición tiene una semántica más específica, como el contenido físico y algunas nociones de propiedad. La composición resulta apropiada cuando cada parte es propiedad de un objeto y cuando las partes no tienen vida independiente separadas de su propietario. Es más útil cuando las partes tienen que ser asignadas e inicializadas en el momento en que se crea su propietario y no sobreviven a la destrucción del mismo. Los atributos de una clase tienen estas propiedades y pueden por ello ser considerados una forma de composición, aunque no se modelan explícitamente como tales. Utilizando la composición, se puede evitar la sobrecarga del gestor de memoria y el peligro de los punteros sueltos u objetos huérfanos. También resulta apropiado para situaciones en las cuales se han aislado una gran cantidad de atributos en una clase distinta por motivos de encapsulamiento y manipulación, pero los atributos realmente pertenecen a la clase principal. Las clases contenedoras utilizadas para implementar asociaciones son también candidatos obvios para partes compuestas, aunque por lo general, deberían ser generadas por un generador de código y no modeladas explícitamente. Obsérvese que una parte compuesta, como por ejemplo una clase contenedora, puede contener referencias (punteros) a partes no compuestas, pero los objetos referenciados no se destruyen cuando se destruye el objeto que los referencia.

agregación compuesta

Véase composición.

agregado

Clase que representa el todo en una asociación de agregación (todo-parte).

alcance

Extensión de un miembro de clasificador, tal como un atributo, operación o nombre de rol; esto es, si representa un valor en cada instancia o bien un valor que comparten todas las instancias

de un clasificador. Cuando se utiliza en solitario sin calificación alguna, indica un alcance de propietario.

Véase creación, alcance de propietario, alcance de destino.

Semántica

El alcance puede ser o bien de propietario o bien de destino.

Alcance de propietario. Indicación de si existe o no una ranura de atributo para cada instancia de una clase, o de si hay una sola ranura con el nombre dado para toda la clase.

instancia	Cada instancia del clasificador posee su propia copia independiente de una ranura de atributo o bien su propio conjunto de objetos asociados. Los valores de una ranura son independientes de los valores que haya en las otras ranuras. Ésta es la situación por defecto. Para una operación, la operación se le aplica a un objeto individual (en una operación normal).
clase	El clasificador en sí posee una copia de la ranura de atributo o del conjunto de objetos asociados. Todas las instancias del clasificador comparten el acceso a una sola ranura. Para una operación, la operación es aplicable a toda la clase, tal como una operación de creación o una operación que proporcione estadísticas relativas a todo un conjunto de instancias. Esta operación no se le puede aplicar a una instancia.

Alcance de destino. Una opción que indica si los valores de un atributo o los valores del destino de una asociación son o no instancias (por defecto) o clasificadores.

instancia	Cada ranura de atributo o cada enlace de la asociación contiene una referencia de una instancia del clasificador destino. El número de enlaces se ve limitado por la multiplicidad. Ésta es la situación por defecto.
clase	Toda ranura de atributo o todo enlace de la asociación contiene una referencia de la clase destino en sí. La información de la asociación queda por tanto determinada en tiempo de modelado y no cambia durante el tiempo de ejecución; de hecho no es necesario almacenarla en todos los objetos. Los enlaces implican a la clase en sí, no a sus instancias. Esto puede ser útil para cierta información de implementación, pero para la mayoría de los propósitos de modelado esta capacidad se puede ignorar.

Discusión

Los atributos o asociaciones de alcance de clase proporcionan valores globales para toda una clase y por tanto deberían utilizarse con cuidado o evitarse por completo, aunque los proporcionan casi todos los lenguajes de programación orientados a objetos. El problema es que implican una información global, lo cual viola el espíritu del diseño orientado a objetos. Además, la información global se vuelve problemática en sistemas distribuidos, porque im-

ne unos accesos centrales en una situación en que los objetos de la clase pueden estar distribuidos en muchas máquinas. En lugar de utilizar las clases como si fuesen objetos con estado, es mejor introducir objetos explícitos para almacenar en ellos cualquier información compartida que pudiera precisarse. Tanto el modelo como los costes se vuelven más evidentes.

Los constructores (operación de creación, operaciones de fábrica) tienen necesariamente un alcance de código del nivel de clase, porque (todavía) no hay ninguna instancia sobre la que puedan operar. Esto es una aplicación correcta y necesaria del alcance de clase. Las otras clases de operaciones cuyo alcance original es del nivel de clase tienen las mismas dificultades que los atributos, a saber, implican una información global centralizada relativa a las instancias de las clases, lo cual no resulta práctico en sistemas distribuidos.

El alcance de destino tiene una utilidad limitada y sólo debería utilizarse en circunstancias especiales generalmente, sólo para propósitos detallados de programación.

alcance de destino

Especificación de si un valor es una instancia o un clasificador.

Véase alcance.

Discusión

El alcance de destino se usa sobre todo para almacenar clases como valores de atributos o como destinos de asociaciones. Tiene una utilidad limitada. La palabra alcance sin aditamentos significa alcance de propietario.

alcance de propietario

Se trata de una indicación de si la característica es aplicable a un objeto individual o si es compartida por toda una clase.

Véase también alcance, alcance de destino.

Semántica

El alcance de propietario indica si existe o no una ranura de atributo distinta para cada instancia de una clase, o si existe una única ranura para toda la clase. Para un operador, el alcance de propietario indica si una operación se aplica a una operación o bien a la clase en sí (tal como sucede en un operador de creación). En algunas ocasiones se denomina simplemente alcance. Los valores posibles son:

instancia

Cada instancia de clasificador posee su propia copia exclusiva de una ranura de atributo. Los valores de una ranura son independientes de los valores de otras ranuras. Ésta es la situación normal.

Para un operador, el operador es aplicable a un objeto individual.

clase

El clasificador en sí posee una copia de la ranura de atributo. Todas las instancias del clasificador comparten el acceso a la única ranura. Si el lenguaje permite clases como objetos reales entonces esto es un atributo de la clase en sí como objeto.

Para un operador, el operador se aplica a toda la clase, como sucede en un operador de creación o en un operador que proporcione estadísticas relativas a todo un conjunto de instancias.

Notación

Los atributos u operadores con alcance de clase se subrayan (Figura 13.14). Los atributos u operadores con alcance de instancia no se subrayan.

Reserva
fecha: Fecha
<u>avanceMax: Hora</u>
<u>crear(fecha: Fecha)</u>
destruir()

Figura 13.14 Atributo y operación con alcance de clase

Discusión

Para una asociación, se diría si la posición de origen de un enlace contiene instancias o clasificadores. Pero esta información se puede especificar como el alcance de destino en la otra dirección, así que el alcance de propietario es innecesario y, por tanto, no se usa para las asociaciones.

alcance original

Indicador de si una ranura es propiedad de una instancia o de una clase.
Véase alcance.

amigo/a

Una dependencia de uso que concede permiso al cliente para tener acceso al proveedor, incluso aunque el cliente no tenga la suficiente visibilidad para tener acceso al proveedor.
Véase también acceso, importar, visibilidad.

Semántica

La dependencia amigo se utiliza para conceder permiso a una clase u operación para usar el contenido de una clase, aunque el permiso fuera insuficiente. Es una excepción explícita a las

reglas normales de permiso entre los elementos afectados. Esta capacidad se debe utilizar cuidadosa y económicamente.

Notación

La dependencia amigo se representa como una flecha de línea discontinua desde la operación o la clase que adquiere el permiso a la clase cuyo contenido se hace disponible; la palabra clave de estereotipo «**friend**» se une a la flecha.

análisis

Etapas de un sistema que captura los requisitos y el dominio del problema. El análisis se centra en lo que hay que hacer, mientras que el diseño se centra en cómo hacerlo. En un proceso iterativo, estas etapas no tienen por qué realizarse de forma secuencial. El resultado de esta etapa se representa mediante modelos del nivel de análisis, especialmente la vista de casos de uso y la vista estática. Contrastar análisis con diseño, implementación y despliegue.

Véase fases de modelado, proceso de desarrollo.

antecesor

Elemento que se encuentra siguiendo una ruta en una o más relaciones padre.

Véase generalización, padre.

argumento

Valor específico de un parámetro.

Véase también ligadura, parámetro, principio de capacidad de sustitución.

Semántica

Una instancia de un mensaje en tiempo de ejecución tiene una lista de valores de los argumentos, cada uno de los cuales es un valor cuyo tipo debe ser consistente con el tipo declarado por el parámetro correspondiente en la declaración de la operación o señal. Un valor es consistente si su clase o tipo de dato es la misma o un descendiente de la declarada como tipo del parámetro. Según el principio de sustitución, se puede utilizar el valor de un descendiente en cualquier sitio donde se haya declarado un antecesor como tipo del parámetro. La implementación de un valor depende del simulador o entorno de ejecución en el que aparece.

En una colaboración o en una máquina de estados pueden aparecer expresiones que implican acciones. Dentro de estas expresiones, las llamadas y los envíos de mensajes requieren especificaciones de argumentos, que son también expresiones. Cuando se evalúan esas expresiones en tiempo de ejecución, deben evaluarse utilizando valores consistentes con los parámetros declarados con los que se corresponden.

Sin embargo, al realizar la ligadura de una plantilla, los argumentos aparecen dentro de un modelo UML durante el propio modelado. En estos casos, los argumentos se representan mediante expresiones en algún lenguaje, normalmente un lenguaje de restricciones o de programación. Los argumentos de las plantillas pueden incluir no sólo valores dato ordinarios u objetos, sino también clasificadores. En este último caso, el tipo de parámetro correspondiente deberá ser **Classifier** (clasificador) o cualquier otro metatipo. El valor de un argumento de plantilla debe fijarse en tiempo de modelado, y no es posible utilizarlo para representar un argumento en tiempo real. No utilice plantillas si no se han fijado los parámetros en tiempo de modelado.

argumento formal

Véase parámetro.

arquitectura

Estructura organizativa de un sistema que incluye su descomposición en partes, conectividad, mecanismos de interacción y principios de guía que proporcionan información sobre el diseño del mismo.

Véase paquete.

Semántica

La arquitectura es el conjunto de decisiones significativas sobre la organización de un sistema software. Incluye la selección de elementos estructurales y las interfaces mediante las que se conectan, la organización a gran escala de los elementos estructurales y la topología de su conexión, su comportamiento en las colaboraciones entre dichos elementos, los mecanismos importantes de que se dispone en el sistema y el estilo arquitectónico que guía su organización. Por ejemplo, la decisión de construir un sistema en dos capas, cada una de las cuales contiene un pequeño número de subsistemas que se comunican de una forma particular es una decisión arquitectónica. La arquitectura de un software no está relacionada sólo con la estructura y el comportamiento, sino también con el uso, la funcionalidad, el rendimiento, la flexibilidad, la reutilización, la facilidad de comprensión, las restricciones y compromisos económicos y tecnológicos y con la estética.

Discusión

Las decisiones arquitectónicas sobre la descomposición de un sistema en partes se pueden capturar utilizando modelos, subsistemas, paquetes, y componentes. Las dependencias entre estos elementos son indicadores clave de la flexibilidad de la arquitectura y de la dificultad de modificar el sistema en el futuro.

Otra parte importante de una arquitectura son los mecanismos que proporciona para construir sobre ella, que deben ser capturados mediante patrones y colaboraciones.

Las decisiones no estructurales se pueden capturar utilizando valores etiquetados.

artefacto

Pieza de información utilizada o producida por un proceso de desarrollo de software, como un documento externo o el producto de un trabajo. Un artefacto puede ser un modelo, una descripción o un software.

asociación

Relación semántica entre dos o más clasificadores que implica la conexión entre sus instancias.

Véase también asociación binaria, asociación *n*-aria, clase asociación, extremo de asociación, generalización de asociaciones.

Semántica

Una asociación es una relación entre dos o más clasificadores que describe conexiones entre sus instancias. Los clasificadores que participan tienen posiciones ordenadas dentro de la asociación. La misma clase puede aparecer en más de una posición dentro de la asociación. Cada instancia de una asociación (denominada enlace) es una tupla (lista ordenada) de referencias a objetos, ya que la asociación es un conjunto de enlaces. Un objeto dado puede aparecer más de una vez en el conjunto de enlaces, o incluso más de una vez (en diferentes posiciones) en el mismo enlace si lo permite la definición de la asociación. Las asociaciones son el “pegamento” que mantiene unido un sistema, pues sin ellas sólo sería un conjunto de clases inconexas.

Estructura

Una asociación tiene un nombre opcional, pero la mayoría de sus descripciones se encuentran en una lista de extremos de asociación, cada uno de los cuales describe cómo participan los objetos de una determinada clase en la asociación. Téngase en cuenta que un extremo de una asociación es simplemente una parte de la descripción de una asociación, y no un concepto semántico o de notación separado.

Nombre. Una asociación puede tener opcionalmente un nombre: una cadena única con respecto a las clases y asociaciones del paquete que la contiene. Una clase asociación es tanto una asociación como una clase, pero sin embargo, las clases y las asociaciones comparten un único espacio de nombres. No es necesario que una asociación tenga un nombre, pues los nombres de roles de sus extremos proporcionan una forma alternativa de distinguir las asociaciones múltiples entre las mismas clases. Por convención, el nombre se lee en el orden en que las clases participantes aparecen en la lista: una Persona trabaja para una Empresa; un Vendedor vende un Coche a un Cliente.

Extremos de asociación. Una asociación contiene una lista ordenada de dos o más extremos de asociación. Por ordenado se entiende que los extremos pueden distinguirse y no son intercambiables. Cada extremo de una asociación define la participación de una clase en una posición dada (rol) en la asociación. La misma clase puede aparecer en más de una posición,

pero las posiciones no son intercambiables, en general. Cada extremo de asociación especifica propiedades que se aplican a los objetos que participan, como cuántas veces puede aparecer en la asociación un objeto simple (multiplicidad). Ciertas propiedades, como la posibilidad de navegación, se aplican sólo a las asociaciones binarias, pero la mayoría se aplican tanto a las binarias como a las n -arias.

Véase extremo de asociación para más detalles.

Instanciación

Un enlace es una instancia de una asociación, que contiene una ranura para cada extremo de la asociación. Cada ranura contiene una referencia a un objeto que es una instancia (directa o indirecta) de la clase especificada como clase del correspondiente extremo de la asociación. Un enlace no tiene identidad aparte de la lista de objetos que contiene. Los enlaces que forman una asociación forman un conjunto donde no pueden existir duplicados. El número de veces que aparece un objeto en un conjunto de enlaces debe ser compatible con la multiplicidad de cada extremo de la asociación. Por ejemplo, si la asociación **EntradasVendidas** conecta muchas entradas con un espectáculo, entonces cada entrada puede aparecer sólo una vez en un enlace, pero cada espectáculo puede aparecer muchas veces, cada una con una entrada diferente.

Se pueden crear y destruir enlaces según avance la ejecución de un sistema, ateniéndose a las restricciones de intercambio de los extremos de asociación. En algunos casos se puede crear o cambiar un enlace de un objeto de uno de los extremos de la asociación pero no del otro extremo. Un enlace se crea a partir de una lista de referencias. No tiene identidad por sí mismo, por lo que carece de sentido hablar de modificar su valor, aunque puede destruirse y crearse otro en su lugar. Un enlace de una clase asociación tiene uno o más valores de atributos además de la lista de objetos que define su identidad, y las operaciones pueden modificar los valores de los atributos mientras preservan las referencias a los objetos participantes.

Notación

Una asociación binaria se representa mediante una ruta continua que conecta los bordes de dos clases (véase Figura 13.15). Una asociación n -aria se representa mediante un rombo conectado

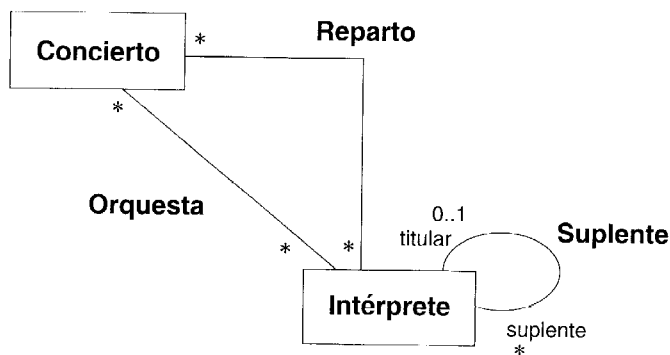


Figura 13.15 Asociaciones

a sus clases participantes mediante rutas (véase Figura 13.19). En la asociación binaria, el rombo se suprime por ser superfluo. Puede haber más de un final de ruta conectada a una misma clase.

Una ruta está formada por uno o más segmentos dibujados con línea continua conectados, normalmente segmentos de recta, aunque están permitidos los arcos y otras curvas, especialmente a la hora de mostrar una auto-asociación (una asociación en la que una clase aparece más de una vez). Los segmentos individuales no tienen semántica significativa. El usuario puede escoger el estilo de línea a utilizar.

Véase ruta.

Los extremos de las rutas tienen adornos que describen la participación de una clase en la asociación. Algunos adornos se representan en el extremo de la ruta, entre el segmento de línea y la caja de la clase. Si hay múltiples adornos, se sitúan secuencialmente comenzando por el extremo de la línea hasta el símbolo de la clase según el orden: flecha de navegación, rombo de agregación/composición y calificador (véase Figura 13.16).

Otros adornos, como las etiquetas de nombre, se sitúan cerca del elemento al que identifican. Los nombres de rol se sitúan cerca del final de la ruta.

Véase extremo de asociación para más detalles sobre la notación de los adornos.

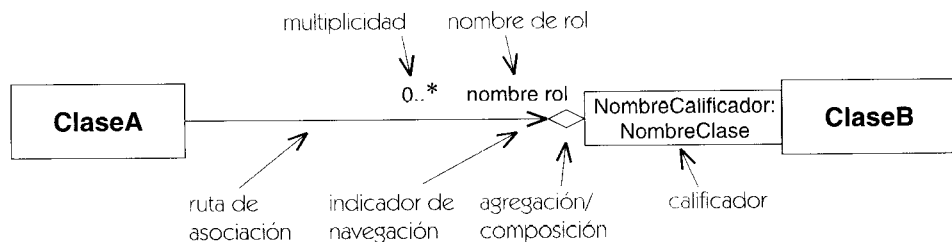


Figura 13.16 Orden de los adornos en un extremo de asociación

Nombre de asociación

El nombre de una asociación se sitúa cerca de la línea de la ruta, pero lo suficientemente lejos del extremo como para que no haya peligro de confusión. El peligro de confusión es meramente visual para las personas, ya que dentro de una herramienta gráfica no existe peligro de ambigüedad en la representación interna, por lo que la herramienta se encarga de determinar la distancia óptima. El nombre de la asociación se puede arrastrar de unos segmentos a otros de una asociación con múltiples segmentos sin que esto tenga ningún significado especial. El nombre de la asociación puede tener al lado un pequeño triángulo relleno que muestre el orden de las clases en la lista. Intuitivamente, la flecha del nombre muestra cómo leer el nombre. En la Figura 13.17, la asociación **TrabajaPara** entre la clase **Persona** y la clase **Empresa** debería tener el triángulo apuntando de **Persona** a **Empresa** y leerse “Persona trabaja para Empresa”. Obsérvese que poner el triángulo en el nombre es un aspecto únicamente de notación que indica el orden en que finaliza la asociación. En el propio modelo, los extremos están inherentemente ordenados, por lo que el nombre del modelo no tiene ni necesita una propiedad de ordenación.

Un estereotipo en la asociación se indica mostrando el nombre del estereotipo entre comillas (« ») enfrente o en lugar del nombre de la asociación. Se puede situar una lista de propiedades después o debajo del nombre de la asociación.

Ejemplo



Figura 13.17 Nombre de asociación

Clase asociación

Una clase asociación se representa asociando un símbolo de clase a la ruta de asociación mediante una línea discontinua. En asociaciones *n*-arias, la línea discontinua se conecta con el rombo de la asociación. Las propiedades “de clase” de la asociación se muestran en el símbolo de clase, mientras que las propiedades “de asociación” de la misma se muestran en la ruta. Observe sin embargo, que la estructura de modelado subyacente es un único elemento, incluso aunque para representarla se utilicen dos estructuras diferentes.

Véase clase asociación para más detalles.

Restricción Xor

La restricción **{xor}** conecta dos o más asociaciones que están conectadas a una misma clase simple (la clase base) en un extremo. Una instancia de la clase base puede participar solamente en una de las asociaciones conectadas por la restricción. Debe respetarse la multiplicidad de la asociación elegida. Si la multiplicidad de alguna asociación incluye la cardinalidad 0, una instancia de la clase base podría no tener enlaces desde la asociación; si no, debería tener uno.

Una restricción **xor** se representa mediante una línea discontinua etiquetada con la cadena de restricción **{xor}** que conecta dos o más asociaciones que tienen una clase en común (Figura 13.18). Los nombres de roles de los extremos de la asociación más lejanos a la clase común deben ser diferentes, lo cual no es más que una utilización predefinida de la notación de restricción que emplea la restricción estándar de solapamiento.

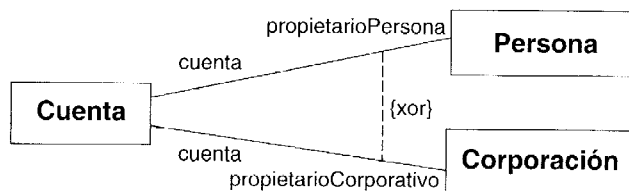


Figura 13.18 Asociación Xor

Discusión

No es necesario ponerle nombre a una asociación. Normalmente, los nombres de rol resultan más convenientes, ya que proporcionan nombres para la navegación y la generación de código a la vez que evitan el problema de saber en qué sentido leer el nombre. Si tiene nombre, debe ser único dentro de su paquete, pero si no lo tiene y hay más de una asociación entre un par (o conjunto) de clases, tienen que existir nombres de rol para distinguir las asociaciones. Si entre dos clases sólo hay una asociación, son suficientes los nombres de las clases para identificarla.

Se puede argumentar que los nombres de asociación son más útiles cuando el concepto tiene un nombre en el mundo real, tales como **Trabajo** o **Matrimonio**. Cuando un nombre de asociación está “dirigido” para ser leído en una determinada dirección, es generalmente más sencillo utilizar nombres de rol, que nunca son ambiguos se lean en la dirección que se lean.

Véase enlace transitorio, donde se discute sobre el modelado de relaciones de instancia que existen sólo durante la ejecución de un procedimiento.

Véase también composición, donde se muestra un ejemplo de generalización en que intervienen dos asociaciones.

Elementos estándar

implicit, persistence, xor.

asociación binaria

Asociación que se produce exactamente entre dos clases.

Véase también asociación, asociación n -aria.

Semántica

Una asociación binaria es una asociación con exactamente dos extremos, que es de lejos la asociación más común de todas. Como un extremo de una asociación binaria tiene otro extremo simple, las asociaciones binarias son particularmente útiles para especificar rutas de navegación entre objetos. Una asociación es navegable en un determinado sentido si puede atravesarse en ese sentido. Otras propiedades, como la multiplicidad, están definidas para asociaciones n -arias, pero son más intuitivas y útiles en las asociaciones binarias.

Notación

Una asociación binaria se representa mediante una ruta continua que conecta dos símbolos de clase. Se pueden asociar adornos a cada extremo y situar el nombre de la asociación cerca de la ruta, pero lo suficientemente lejos de los extremos como para que no haya confusiones con los nombres de rol. La notación de una asociación binaria es la misma que la de una asociación

n -aria excepto en que se suprime el rombo central. Sin embargo, las asociaciones binarias pueden tener adornos que no son aplicables a las asociaciones n -arias, como la posibilidad de navegación.

Para más detalles, véase asociación.

asociación de comunicación

Asociación que describe una relación de comunicación entre instancias de los elementos conectados. En una vista de despliegue, es una asociación entre nodos que implica una comunicación. En un modelo de casos de uso, una asociación entre un caso de uso y un actor constituye una asociación de comunicación.

Véase actor, caso de uso.

asociación n -aria

Denota una asociación entre tres o más clases. Por contrastar con: asociación binaria.

Semántica

Toda instancia de la asociación es una n -tupla de valores pertenecientes a las respectivas clases. Una clase puede aparecer en más de una posición de la asociación. Una asociación binaria es un caso especial que posee su propia notación simplificada y ciertas propiedades adicionales (tales como la navegabilidad) que carecen de sentido (o son inútilmente complicadas) para una asociación n -aria.

La multiplicidad de las asociaciones n -arias se puede especificar pero resulta menos evidente que la multiplicidad binaria. La multiplicidad del extremo de una asociación representa el número potencial de valores que puede haber en ese extremo, cuando están fijados los valores de los otros $n-1$ extremos. Obsérvese que esta definición es compatible con la multiplicidad binaria.

La agregación (incluyendo la composición) sólo tiene significado para las asociaciones binarias. Una asociación n -aria no puede contener el símbolo de agregación ni el de composición en ningún rol.

No existe diferencia semántica entre una asociación binaria y una asociación n -aria con dos extremos, independientemente de la representación. Una asociación con dos extremos se considera una asociación binaria y una que tenga más de dos extremos se considera asociación n -aria.

Notación

Una asociación n -aria se representa mediante un rombo grande (esto es, grande en comparación con el terminador de una ruta), existiendo una ruta desde el rombo hasta cada una de las clases participantes. El nombre de la asociación (si existe) se muestra junto al rombo. Pueden

aparecer adornos junto al extremo de cada ruta, tal como sucedería con una asociación binaria. Se puede indicar la multiplicidad, pero no se permiten los calificadores ni las agregaciones.

Se puede asociar un símbolo clase de asociación al rombo mediante una línea discontinua. Esto indica una asociación n -aria que posee atributos, operaciones y/o asociaciones.

Ejemplo

La Figura 13.19 muestra el registro de un equipo en las distintas temporadas de un determinado guardameta. Se supone que el portero puede ser traspasado durante la temporada y por tanto puede tener un registro con distintos equipos. En un libro de registros, cada enlace sería una línea diferente.

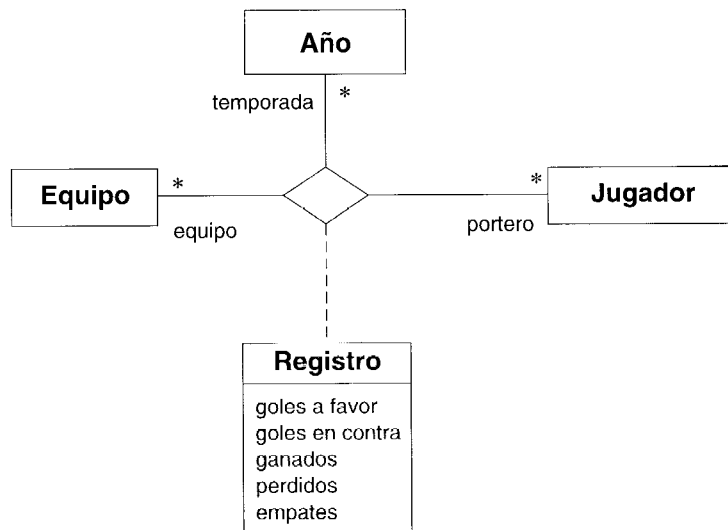


Figura 13.19 Una asociación ternaria que es además una clase asociación

Reglas de estilo

Normalmente, las líneas se dibujan saliendo de los vértices del rombo o desde el punto medio de una cara.

Discusión

En una asociación n -aria, la multiplicidad se define con respecto a los otros $n-1$ extremos. Por ejemplo, dada una asociación ternaria entre clases (A, B, C), la multiplicidad del extremo C indica el número de objetos C que pueden aparecer asociados a una pareja particular de objetos A y B. Si la multiplicidad de esta asociación es (muchos, muchos, uno) entonces para cada posible pareja (A, B) existe un único valor de C. Para una pareja dada (B, C) puede haber

muchos valores de A, sin embargo; de modo individual muchos valores de A, B y C podrían participar en la asociación. En una asociación binaria esta regla se reduce a la multiplicidad de cada extremo, que se define con respecto al otro extremo.

No tiene sentido definir la multiplicidad con respecto a un solo extremo (tal como han propuesto algunos autores) porque la multiplicidad sería muchos para cualquier asociación *n*-aria significativa. De no ser así, la asociación se podría fragmentar en una asociación binaria entre la clase única y una clase asociación que incluyera todas las clases restantes, mejorando así tanto la precisión como la eficiencia de la implementación. En general, lo mejor es evitar las asociaciones *n*-arias porque las asociaciones binarias son más fáciles de implementar y hacen posible la navegación. En general, las asociaciones *n*-arias sólo son útiles cuando se necesitan todos los valores para determinar un enlace de forma única. Una asociación *n*-aria se va a implementar en casi todas las ocasiones como una clase entre cuyos atributos se cuentan punteros de los objetos participantes. La ventaja de modelarla como una asociación es la restricción de que no puede haber enlaces duplicados dentro de una asociación.

Considere el ejemplo de un alumno que asiste a un curso de un profesor a lo largo de un curso (Figura 13.20). El alumno no va a seguir el mismo curso de más de un profesor, pero un alumno puede asistir a más de un curso de un único profesor, y el profesor puede impartir más de un curso. Las multiplicidades se muestran en el diagrama. La multiplicidad del **Profesor** es opcional (0..1); las otras multiplicidades son muchas (0..*).

Todo valor de multiplicidad es relativo a una pareja de objetos de otros extremos. Para toda pareja (curso, alumno) existe cero o un profesor. Para toda pareja (alumno, profesor) existen muchos cursos. Para toda pareja (curso, profesor) existen muchos alumnos.

Obsérvese que si esta asociación se materializa en una clase entonces sería posible tener más de una copia de la misma combinación (alumno, curso, profesor) lo cual no resulta deseable.

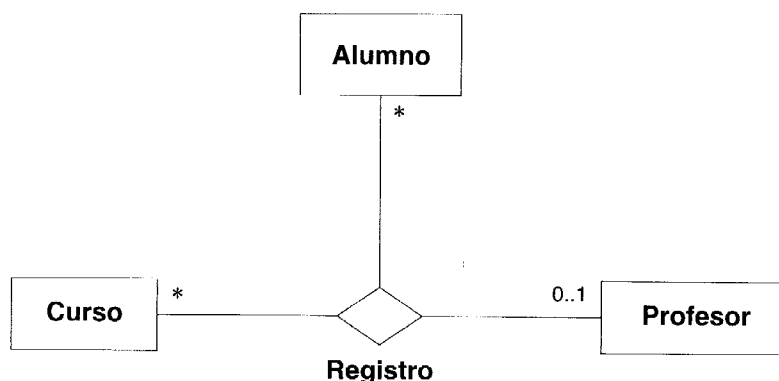


Figura 13.20 Multiplicidad en una asociación *n*-aria

atómico/a

Acción u operación cuya ejecución debe completarse como un todo, por lo que no puede ejecutarse parcialmente ni ser finalizada por un evento externo. Normalmente, las operaciones

atómicas son pequeñas y sencillas, como asignaciones, cálculos aritméticos sencillos u operaciones de tipo cadena. Un cómputo atómico ocurre en un punto definido de la secuencia de ejecución.

Véase también acción, actividad, ejecución a completar.

Semántica

Globalmente, el sistema puede realizar muchas acciones simultáneamente. Cuando decimos que una acción es atómica, eso no implica que el sistema completo sea atómico. El sistema puede procesar interrupciones del hardware y compartir el tiempo entre diversas acciones, ya que una acción es atómica sólo dentro de su hilo de control. Una vez iniciada, debe completar su ejecución sin interactuar con otras acciones que estén activas al mismo tiempo. El sistema puede procesar interrupciones y eventos, pero eso no debe afectar a la operación atómica, ya que dichas acciones no deberían usarse como un mecanismo para transacciones largas. Su duración debería ser breve comparada con el tiempo de respuesta necesario para responder a eventos externos porque, si no, el sistema podría ser incapaz de responder a tiempo.

atributo

Un atributo es la descripción de una ranura con nombre de un tipo especificado en una clase; cada objeto de la clase tiene un valor independiente para el atributo.

Semántica

Un atributo es una ranura con nombre dentro de un clasificador que describe los valores que pueden tener las instancias del clasificador. Todas las instancias del clasificador o de uno de sus descendientes tienen una ranura que guarda un valor del tipo dado. Todas las ranuras son distintas e independientes de las demás (excepto en el caso de los atributos con alcance de clase, que se describen más adelante). Cuando se lleva a cabo una ejecución, el valor de una ranura dentro de una instancia puede remplazarse por otro del mismo tipo, ya que los atributos son modificables.

Un clasificador forma un espacio de nombres para sus atributos, aunque también forman parte del espacio de nombres los pseudoatributos, los nombres de rol de las asociaciones que salen del clasificador y los discriminadores de generalizaciones en las que participe el clasificador.

Estructura

Un atributo tiene los siguientes componentes principales, que se describen con más detalle en sus propias entradas:

alcance de propietario En una clase, el valor descrito por un atributo puede ser distinto en cada objeto o compartido por todos los objetos. En el primer caso,

se trata de un atributo con alcance de instancia, mientras que en el último se trata de un atributo con alcance de clase. La mayoría de los atributos tienen alcance de instancia, esto es, mantienen información de estado acerca de un objeto en concreto. Los atributos con alcance de clase guardan información sobre una clase entera, ya que hay un único valor de la ranura para toda la clase.

Mientras que un atributo con alcance de instancia es una descripción de un valor sin existencia hasta que no se instancia el objeto, un atributo con alcance de clase representa la declaración de un valor discreto individual que existe a lo largo de toda la vida de un sistema.

alcance destino	El valor de un atributo puede ser una instancia o una clase. En el primer caso sería un alcance de instancia (valor por defecto), mientras que en el segundo es un alcance de clase, caso raro que normalmente implica alguna forma de metamodelado.
multiplicidad	El posible número de valores del atributo que pueden existir simultáneamente. El valor más común, “exactamente uno” denota un atributo escalar. El valor “cero o uno” denota un atributo con un valor opcional. La no existencia de valor es distinta de cualquier valor del dominio del tipo del atributo (en otras palabras, la ausencia de valor es distinta del valor cero, pues significa conjunto vacío). Otras multiplicidades denotan atributos que pueden tener múltiples valores. Si la multiplicidad no es un entero simple, es porque el número de valores del atributo puede variar. La multiplicidad “muchos” denota un conjunto ilimitado de valores.
nombre	El nombre de un atributo, una cadena que debe ser única dentro de la clase y sus antecesores. También debe ser única entre los nombres de los roles de asociación alcanzables desde la clase.
tipo	Designa una clase o tipo de dato del que son instancias los valores de la ranura. Un valor puede ser una instancia de un descendiente de la clase o tipo de dato dado.
valor inicial	Expresión que indica el valor de un atributo en un objeto justo después de haber sido inicializado. La expresión es una cadena de texto, junto con el nombre del lenguaje utilizado para evaluar la expresión. La expresión se evalúa en el contexto del lenguaje cuando se instancia el objeto. Véase expresión para más detalles. El valor inicial es opcional; si no lo hay, el modelo estático no especifica el valor para los objetos nuevos, aunque otra parte del modelo global puede proporcionar dicha información.

Obsérvese que un procedimiento de inicialización explícito, como un constructor, puede reemplazar una expresión de valor inicial.

El valor inicial de un atributo con alcance de clase se utiliza para inicializarlo una vez al principio de la ejecución. UML no especi-

	ca el orden relativo de inicialización de los diferentes atributos con alcance de clase.						
variabilidad	Indica si el valor de la ranura puede cambiar tras la inicialización. Se utiliza una enumeración cuyo valor por defecto es changeable . Los posibles valores son: <table> <tr> <td>changeable</td><td>(modificable): no hay restricciones para modificarlo. Valor por defecto.</td></tr> <tr> <td>addOnly</td><td>(sólo añadir): para atributos con multiplicidad mayor que uno, se pueden añadir valores adicionales al conjunto de valores del atributo, pero una vez creado, un valor no puede ser modificado ni eliminado.</td></tr> <tr> <td>frozen</td><td>(congelado): el valor no se puede modificar una vez inicializado el objeto. No se pueden añadir valores adicionales a un conjunto de valores.</td></tr> </table>	changeable	(modificable): no hay restricciones para modificarlo. Valor por defecto.	addOnly	(sólo añadir): para atributos con multiplicidad mayor que uno, se pueden añadir valores adicionales al conjunto de valores del atributo, pero una vez creado, un valor no puede ser modificado ni eliminado.	frozen	(congelado): el valor no se puede modificar una vez inicializado el objeto. No se pueden añadir valores adicionales a un conjunto de valores.
changeable	(modificable): no hay restricciones para modificarlo. Valor por defecto.						
addOnly	(sólo añadir): para atributos con multiplicidad mayor que uno, se pueden añadir valores adicionales al conjunto de valores del atributo, pero una vez creado, un valor no puede ser modificado ni eliminado.						
frozen	(congelado): el valor no se puede modificar una vez inicializado el objeto. No se pueden añadir valores adicionales a un conjunto de valores.						
visibilidad	Indica si el atributo es visible desde otras clases. Se representa mediante una enumeración donde pueden aparecer las palabras clave public , private o protected . Se pueden añadir valores adicionales para modelar ciertos lenguajes de programación.						

Notación

Un atributo se representa mediante una cadena de texto que puede dividirse en varias propiedades. La sintaxis por defecto es:

$$\overline{\text{«estereotipo»}}_{opc} \text{ visibilidad}_{opc} \text{ nombre multiplicidad}_{opc} : \text{tipo}_{opc}$$

$$= \text{valor-inicial}_{opc} \{ \text{cadena-de-propiedades} \}_{opc}$$

Visibilidad. La visibilidad se representa mediante una marca de puntuación, aunque también se puede representar utilizando una palabra clave dentro de la cadena de propiedades, que se utiliza sobre todo para incluir opciones definidas por el usuario o dependientes del lenguaje. Las opciones predefinidas son:

+ (public)	Cualquier clase que vea la clase ve también sus atributos.
# (protected)	Sólo la propia clase o sus descendientes pueden ver los atributos.
– (private)	Sólo la propia clase puede ver los atributos.

Nombre. El nombre se representa mediante un identificador de tipo cadena.

Tipo. El tipo se representa mediante una expresión de cadena que denota un clasificador. El nombre de una clase o tipo de datos es una expresión legítima de tipo cadena que indica que los valores de un atributo deben ser de un tipo dado. La sintaxis adicional del tipo depende del

lenguaje de la expresión. Cada lenguaje tiene una sintaxis de construcción de nuevos tipos de datos a partir de los datos existentes. Por ejemplo, C++ tiene sintaxis para los punteros, para los arrays y para las funciones. Ada también tiene sintaxis para los subrangos. El lenguaje de la expresión es parte del modelo interno, pero normalmente no se indica en un diagrama, ya que se supone conocida en todo el diagrama u obvio a partir de su sintaxis.

La cadena de tipo se puede suprimir (pero sigue presente en el modelo).

Multiplicidad. La multiplicidad se representa mediante una expresión de multiplicidad encerrada entre corchetes y situada después del nombre del atributo. Si la multiplicidad es “exactamente uno”, se puede omitir la expresión, incluyendo los corchetes. Esto indica que cada objeto tiene exactamente una ranura para guardar un valor del tipo especificado. Si no es así, debe mostrarse la multiplicidad. Véase multiplicidad, donde hay una completa discusión sobre la sintaxis. Por ejemplo:

colores [3]:Saturación	Un array de 3 saturaciones
puntos [2..*]:Punto	Un array de dos o más puntos

Obsérvese que una multiplicidad de 0..1 abre la posibilidad de valores nulos (la ausencia de valor como oposición a un valor concreto dentro del rango). Un valor nulo no es un valor del dominio de la mayoría de los tipos de datos: extiende el dominio con un valor extra que se encuentra fuera del mismo. Sin embargo, en el caso de los punteros el valor nulo es a menudo parte de la implementación (aunque, incluso entonces, se usa normalmente por convenio: por ejemplo el valor 0 en C o C++ es un convenio de direccionamiento de memoria). La siguiente declaración permite distinguir entre el valor *null* y la cadena vacía, una distinción presente en C++ y en otros lenguajes:

nombre [0..1]:String	Si no tiene nombre, es un valor nulo.
----------------------	---------------------------------------

Valor inicial. El valor inicial se representa mediante una cadena. El lenguaje de evaluación normalmente no se indica explícitamente, aunque está presente en el modelo. Si no hay valor inicial, se omiten tanto la cadena como el signo igual. Si la multiplicidad del atributo incluye el valor 0 (es decir, opcional) y no se le ha dado explícitamente ningún valor, el atributo comienza teniendo un valor nulo (cero repeticiones).

Variabilidad. El valor de modificación se representa mediante una palabra clave que representa el valor elegido. Si no se elige ningún valor, se toma por defecto **changeable**.

Valor etiquetado. Un atributo puede tener asociados cero o más valores etiquetados (como cualquier elemento del modelo). Cada valor etiquetado se representa de la forma etiqueta = valor, donde etiqueta es el nombre de una etiqueta y valor es un valor literal. Los valores etiquetados se incluyen con las palabras clave de las propiedades como una lista entre corchetes de propiedades separadas por comas.

Alcance. El alcance de clase de un atributo se representa subrayando la cadena que expresa el tipo y nombre. Si no está subrayado, el atributo tiene alcance de instancia. La justificación de la notación es que un atributo con alcance de clase es un valor en el sistema en ejecución, al igual que un objeto es un valor de instancia, por lo que ambos deben subrayarse.

atributo-con-alcance-de-clase

La Figura 13.21 muestra la declaración de algunos atributos.

+tamaño: Área = (100,100)	público, valor inicial
#visibilidad: Boolean = invisible	protegido, valor inicial
+tamaño-por-defecto: Rectángulo	público
tamaño-máximo: Rectángulo	alcance de clase
-xptr: XWindowPtr {requisito = 4.3}	privado, valor etiquetado

Figura 13.21 Atributos

Opciones de presentación

Sintaxis del lenguaje de programación. La sintaxis de la cadena de un atributo puede ser la de un lenguaje de programación, como C++ o Smalltalk. Se pueden incluir en la cadena propiedades etiquetadas específicas.

Reglas de estilo

Los nombres de atributo se representan con un tipo de letra normal.

Discusión

Se utiliza una sintaxis similar para representar calificadores, parámetros de plantillas, parámetros de operaciones, y otros, aunque algunos de ellos omiten ciertos términos.

Obsérvese que un atributo es equivalente semánticamente a una asociación de composición, aunque el uso y la intención son totalmente diferentes. Utilice los atributos para tipos de datos, esto es, para valores sin identidad. Utilice las asociaciones para clases, o lo que es lo mismo, para valores con identidad. La razón es que en los objetos con identidad es importante ver la relación en ambas direcciones, mientras que en los tipos de datos, el tipo normalmente está subordinado al objeto y no sabe que existe.

Elementos estándar

persistence.

autotransición

Transición en la cual el estado origen y el estado destino son el mismo. Se considera un cambio de estado. Cuando se dispara, se sale del estado origen y se vuelve a entrar en él, de tal modo que se invocan las acciones de entrada y las de salida. No es equivalente a una transición interna, en la cual no se produce un cambio de estado.

become (se convierte en)

Tipo de dependencia de flujo, utilizada en una interacción, en la cual el objeto destino representa una nueva versión del objeto origen y a partir de entonces lo reemplaza.

Véase también clase en un estado, copia, localización.

Semántica

Una dependencia *become* es un tipo de dependencia de flujo que representa la derivación de un objeto a partir de otro dentro de una interacción. Después de ejecutarse un flujo *become*, el nuevo objeto reemplaza al original dentro del cómputo. Normalmente no es necesario utilizar esta relación para representar únicamente un cambio en el valor de un objeto. Por otra parte, esta relación es útil para representar un cambio cualitativo en un objeto, como un cambio de estado, un cambio de clase o un cambio de localización. En estas situaciones, el modelo contiene dos versiones del objeto, pero la relación *become* muestra que son realmente el mismo objeto, esto es, que tienen la misma identidad.

Una transición *become* dentro de una interacción puede tener un número de secuencia para indicar cuándo sucede con respecto a otras acciones.

Notación

Un flujo *become* se representa mediante una flecha discontinua con el estereotipo «become» que sale de la primera versión del objeto y apunta a la última. En una interacción, la flecha puede tener un número de secuencia cuando el cambio se produce con relación a otras acciones. Las transiciones *become* pueden aparecer en diagramas de colaboración, en diagramas de secuencia y en diagramas de actividades.

En un diagrama de actividades, la transición *become* se puede representar mediante una flecha discontinua desde o hacia el símbolo de flujo de objetos. Se puede omitir la palabra clave *become*.

Ejemplo

La Figura 13.22 muestra una orden para abrir un icono de directorio cerrado en un escritorio, seguido por otra que ordena los elementos del recién abierto directorio. El directorio se muestra dos veces como objeto de una clase en un estado, con una transición *become* entre las dos versiones.

La Figura 13.142 muestra un diagrama de despliegue en el que un objeto migra entre dos nodos.

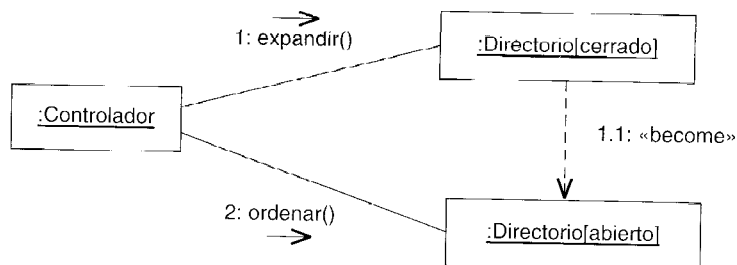


Figura 13.22 Flujo become

bien formado

Denota un modelo que está construido correctamente, que satisface todas las reglas predefinidas y establecidas por el modelo. Este modelo tiene una semántica significativa. Un modelo que no sea bien formado recibe el nombre de mal formado.

bifurcación

Elemento de una máquina de estados en el que un único disparador tiene más de un posible resultado, cada uno con su propia condición de guarda.

Véase también división, estado de unión o conjunción, unión.

Semántica

Si un mismo evento puede tener diferentes efectos dependiendo de las distintas condiciones de guarda, puede modelarse entonces como transiciones separadas con el mismo evento disparador. Sin embargo, en la práctica es conveniente permitir que un disparador simple dirija múltiples transiciones. Esto es especialmente cierto en el caso tan común en que las condiciones de guarda cubren todas las posibilidades, caso en el que se garantiza que una ocurrencia del evento disparará una de las transiciones. Una *bifurcación* es una parte de la transición que divide la ruta de la misma en dos o más segmentos, cada uno de ellos con una condición de guarda. El evento disparador se sitúa en el segmento común de la transición (el primer segmento). La salida de un segmento de una bifurcación puede conectarse con la entrada de otra bifurcación formando un árbol, de forma que cada ruta a través del árbol represente una transición diferente. La conjunción de todas las condiciones de una ruta de una transición es equivalente a una condición simple que se evalúa antes de que se dispare la transición. Una transición se dispara en un único paso, a pesar de su apariencia de conjunto de bifurcaciones, ya que el árbol no es más que un convenio de notación.

En un grafo de actividades, las actividades que salen de un estado de actividad son, generalmente, transiciones de finalización, esto es, carecen de eventos disparadores explícitos, y se disparan implícitamente cuando finaliza la actividad del estado. Si existen condiciones de guarda o bifurcaciones, es importante que cubran todas las posibilidades para que se dispare alguna transición, pues si no se congelaría la ejecución del grafo de actividades, ya que las transiciones de salida nunca volverían a estar disponibles.

Notación

Una bifurcación se puede representar repitiendo un evento disparador en los arcos de muchas transiciones con diferentes condiciones de guarda. También con transiciones de finalización, como en los diagramas de actividades.

Sin embargo, es más conveniente representar las bifurcaciones utilizando un rombo al que se conecta la punta de flecha de una transición. La flecha de la transición se etiqueta con el evento disparador, si lo hay, pero no debería llevar acciones asociadas. Las acciones irían en el segmento final de la transición.

De un rombo pueden salir dos o más flechas, cada una de las cuales se etiqueta con una condición de guarda. Se puede utilizar la palabra reservada **else** como condición de guarda, que será verdadera cuando las demás condiciones explícitas de guarda sean falsas. El final de una

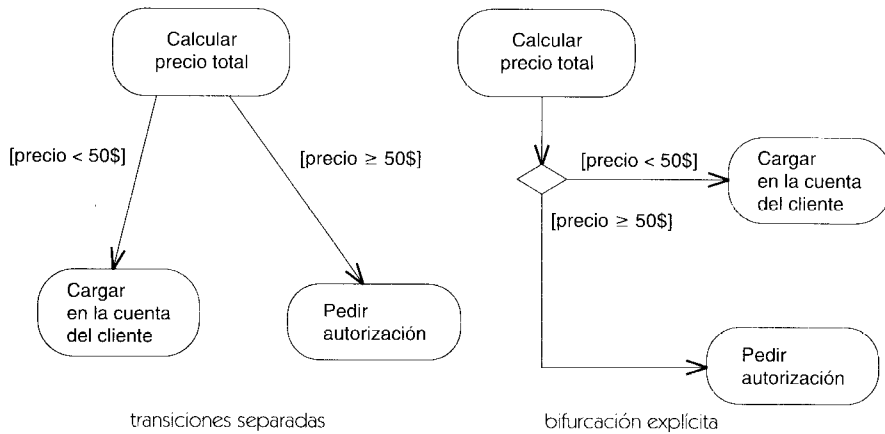


Figura 13.23 Dos formas de representar una bifurcación

flecha se puede conectar con otra bifurcación o con otro estado, y cuando se conecte a un estado, podrá llevar una acción asociada.

El efecto de árbol de bifurcaciones es el mismo que el que se produce expandiendo el árbol en un arco de transición separado para cada ruta del árbol, donde todos comparten el mismo evento disparador, pero cada uno de los cuales tiene su propia conjunción de condiciones de guarda, acciones y estados finales. La Figura 13.23 muestra dos formas de representar la misma situación.

Obsérvese que se puede utilizar también el símbolo del rombo para la reunificación (operación inversa de la bifurcación), donde dos rutas separadas se unen, tal y como se muestra en la Figura 13.24. En el caso de la reunificación habrá dos o más flechas de entrada y una sola de salida. No son necesarias las condiciones de guarda.

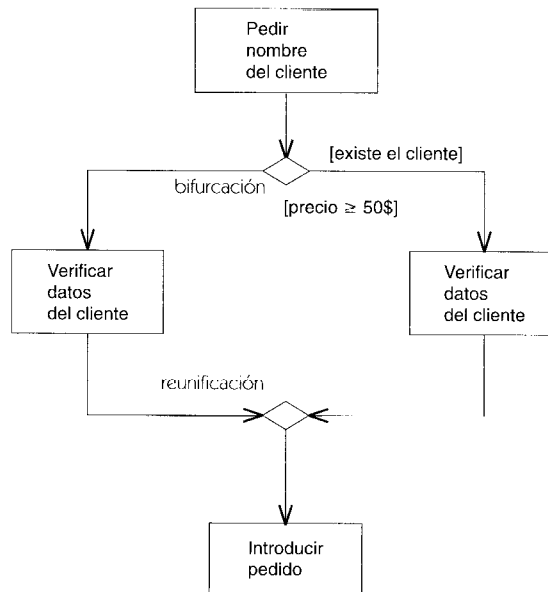


Figura 13.24 Bifurcación y reunificación

bind (ligar)

Palabra clave para expresar una dependencia de ligadura en la notación.

Véase ligadura.

booleano/a

Enumeración cuyos valores son **true** y **false**.

cadena

Secuencia de caracteres de texto. Los detalles de la representación de cadenas dependen de la implementación y pueden incluir conjuntos de caracteres que admitan gráficos y caracteres internacionales.

Semántica

Hay muchas propiedades semánticas, especialmente los nombres, que tienen una cadena como valor. Una cadena es una secuencia de caracteres pertenecientes a algún conjunto de caracteres adecuado, que se emplea para visualizar información relativa al modelo. Entre los juegos de caracteres se pueden incluir alfabetos y caracteres no latinos. UML no especifica la codificación de cadenas pero supone que la codificación es suficientemente general para permitir cualquier utilización razonable. En principio, la longitud de las cadenas debería ser ilimitada; toda limitación práctica debería ser suficientemente grande para que no resulte restrictiva. Las cadenas deberían incluir la posibilidad de admitir mostrar caracteres de diferentes idiomas humanos. Los identificadores (los nombres) deberían estar formados únicamente por caracteres de un conjunto finito de caracteres. Los comentarios y clases similares de cadenas descriptivas sin contenido semántico puede contener otras clases de medios de elementos, tales como diagramas, grafos, imágenes o segmentos de vídeo y otros tipos de documentos incrustados.

Notación

Una cadena gráfica es un elemento primitivo de notación con cierta flexibilidad de implementación. Se supone que se trata de una secuencia lineal de caracteres escrita en algún idioma, con la posible inclusión de documentos incrustados de distintas clases. Resulta deseable admitir la utilización de diferentes idiomas humanos, pero se dejan los detalles a las herramientas de edición para que sean éstas las que los implementen. Las cadenas gráficas pueden ir en líneas individuales, en listas o bien pueden ser etiquetas asociadas a otros símbolos.

Las cadenas se utilizan para mostrar propiedades semánticas que tienen cadenas como valores y también se emplean para codificar los valores de otras propiedades semánticas para su visualización. La correspondencia entre cadenas semánticas y cadenas de notación es directa.

La correspondencia de otras propiedades con las cadenas de notación está controlada por las gramáticas, que se describen en los artículos correspondientes a los distintos elementos. Por ejemplo, la notación de visualización para los atributos codifica el nombre, valor, visibilidad y alcance en una única cadena de visualización.

Es posible realizar extensiones no canónicas de las codificaciones —por ejemplo, se podría visualizar un atributo empleando la notación de C++—. Algunas de estas codificaciones pueden perder información del modelo, sin embargo, las herramientas deberían admitirlas como opciones seleccionables por el usuario, manteniendo ciertamente la notación canónica de UML.

El estilo y tamaño del tipo de letra son marcadores gráficos, que normalmente serán independientes de la cadena en sí. Se pueden codificar para distintas propiedades del modelo, algunas de las cuales se sugieren en este documento; otras se dejan al albedrío de la herramienta o del usuario. Por ejemplo, la cursiva muestra clases y operaciones abstractas y el subrayado muestra características con alcance de clase.

Las herramientas pueden tratar de distintas maneras las cadenas más largas; se pueden truncar hasta un tamaño dado, se puede hacer un salto de línea automático y se pueden insertar barras de desplazamiento. Se supone que existe alguna manera de obtener la cadena completa si se desea.

calificador

Denota una ranura de un atributo o lista de atributos en una asociación binaria, en la cual los valores de los atributos seleccionan un único objeto relacionado o un conjunto de objetos relacionados dentro de todo el conjunto de objetos relacionados con un objeto por esa asociación. Se trata de un índice para recorrer una asociación.

Véase clase asociación, extremo de asociación.

Semántica

Toda asociación binaria hace corresponder a un objeto un conjunto de objetos relacionados. En algunas ocasiones resulta deseable seleccionar un objeto del conjunto, proporcionando un valor que distingue a los objetos del conjunto. Este valor podría ser un atributo de la clase destino. En general, sin embargo, el valor selector podría ser parte de la asociación en sí, un atributo de la asociación cuyo valor es proporcionado por el creador cuando se añade un nuevo enlace a la clase asociación. Un atributo como éste, en una asociación binaria, recibe el nombre de calificador. Un objeto, junto con un valor de calificador, determina un objeto único relacionado con él o bien (con menos frecuencia) un subconjunto de objetos relacionados. El valor califica la asociación. En un contexto de implementación, a estos atributos se les da el nombre de valores de índice.

Los calificadores se utilizan para seleccionar un objeto u objetos del conjunto de objetos relacionados con un objeto (que se denomina *objeto calificado*) mediante una asociación (Figura 13.25). El objeto seleccionado por el valor del calificador se denomina *objeto destino*.

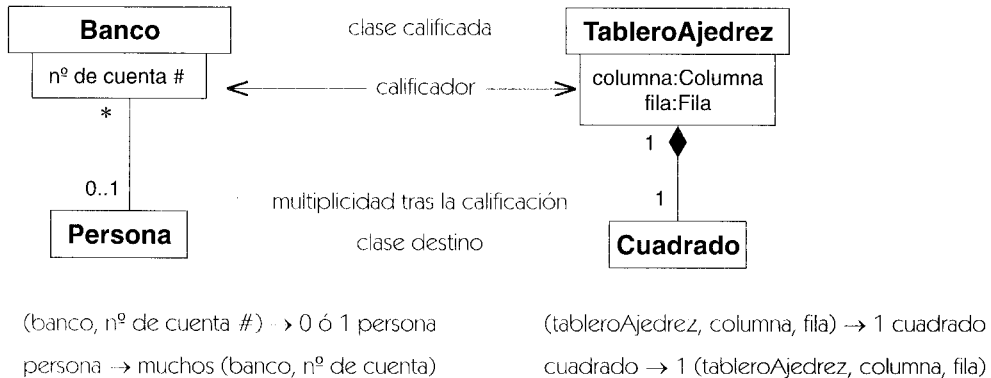


Figura 13.25 Asociaciones calificadas

Un calificador siempre actúa sobre una asociación cuya multiplicidad es *muchos* en la dirección del destino. En el caso más sencillo, cada valor del calificador selecciona un único objeto del conjunto destino de objetos relacionados. En otras palabras, un objeto calificado y un valor de calificador producen un único objeto destino relacionado. Dado un objeto calificado, cada valor del calificador se corresponde con un único objeto destino.

Hay muchas clases de nombres que son calificadores. Estos nombres dentro de un contexto se corresponden con un único valor. El objeto calificado proporciona el contexto y el objeto destino es el resultado. Toda ID u otro código único es un calificador; su propósito es seleccionar un valor de modo único. Las matrices se pueden modelar como asociaciones calificadas. La matriz es el objeto calificado, el índice de la matriz es el calificador y el elemento de la matriz es el objeto destino. Para una matriz, el tipo calificador es un intervalo de enteros.

Se puede emplear un calificador en una expresión de navegación para seleccionar un subconjunto de objetos relacionados con un objeto a través de una asociación, a saber, aquellos que tengan un valor concreto para el valor del atributo calificador o lista de valores. El calificador hace las veces de selector dentro del conjunto de objetos relacionados por la asociación. Descompone el conjunto en subconjuntos por valores del calificador. En la mayoría de los casos, el propósito del calificador es seleccionar un objeto único dentro del conjunto de objetos relacionados de tal modo que una asociación calificada se comporta como una tabla de búsqueda.

Estructura

Calificador. Un atributo calificador es una parte opcional del extremo de una asociación binaria. El calificador califica a la clase asociada al extremo de la asociación. Un objeto de la clase y un valor del calificador seleccionan un objeto o conjunto de objetos de la clase situada al otro extremo de la asociación binaria. Es posible que los dos extremos de la asociación binaria tenga calificadores, pero no suele suceder.

Un calificador es un atributo o lista de atributos de una asociación. Cada atributo posee nombre y tipo pero no valor inicial, porque los calificadores no son objetos independientes; el valor de cada calificador debe ser explícito siempre que se añade un enlace a la asociación.

Los calificadores no se utilizan en asociaciones *n*-arias.

Multiplicidad. La multiplicidad de la relación calificada se sitúa en el extremo opuesto de la asociación binaria respecto al calificador. (La regla mnemotécnica es que la clase calificada y el calificador, juntos, forman un valor compuesto que está relacionado con la clase destino.) En otras palabras, el calificador se asocia al extremo “próximo” de la asociación y la multiplicidad y el nombre de rol se asocian al extremo “lejano”.

La multiplicidad asociada al extremo destino de la asociación denota el número de objetos destino que podrían ser seleccionados por una pareja (objeto origen, valor de calificador). Entre los valores comunes de multiplicidad se cuentan **0..1** (se puede seleccionar un único valor pero no todo posible valor del calificador selecciona un valor necesariamente), **1** (todo posible valor del calificador selecciona un único objeto destino, luego el dominio de valores del calificador debe ser finito) y ***** (el valor del calificador es un índice que descompone el objeto destino en subconjuntos).

En la mayoría de los casos, la multiplicidad es cero-o-uno. Esta opción significa que un objeto y un calificador pueden producir, como máximo, un único objeto relacionado. Una multiplicidad de uno significa que todo posible valor del calificador produce exactamente un objeto. Evidentemente, esto requiere que el tipo del calificador sea un dominio finito (al menos en la implementación para un computador). Esta multiplicidad puede ser útil para hacer corresponder tipos finitos enumerados; por ejemplo, un **Píxel** calificado por un **ColorPrimario** (enumeración de rojo, verde y azul) produciría el triplete de valores rojo-verde-azul para todos los píxeles de una imagen.

La multiplicidad de una asociación no calificada no se enuncia explícitamente. Pero suele suponerse que es muchos, o por lo menos mayor que uno. En caso contrario, no habría necesidad de tener un calificador.

Una multiplicidad de muchos en una asociación calificada no tiene un impacto semántico significativo, porque el calificador no reduce la multiplicidad del conjunto destino. Esta multiplicidad representa una sentencia de diseño consistente en que debe proporcionarse un índice para recorrer la asociación. En tal caso, el calificador descompone el conjunto de objetos destino en subconjuntos. Semánticamente, esto no añade nada más allá de tener un atributo de asociación, que también descompone (implícitamente) los enlaces. La connotación de diseño de un calificador en un modelo de diseño es que el recorrido debería ser eficiente, esto es, que no debe requerir una búsqueda lineal entre todos los valores destino. Normalmente se implementa mediante algún tipo de tabla de búsqueda. Un índice de una base de datos o de una estructura de datos se modela correctamente como un calificador.

En la dirección inversa a través de una asociación calificada (esto es, yendo desde la clase destino hacia el objeto calificado), la multiplicidad indica el número de parejas (objeto calificado, calificador) que se pueden relacionar con un objeto destino, no el número de objetos calificados. En otras palabras, si varias parejas (objeto calificado, calificador) se corresponden con el mismo objeto destino, entonces la multiplicidad inversa será muchos. Una multiplicidad inversa de uno entre destino y calificador significa que existe exactamente un emparejamiento de objeto calificado y valor calificador que está relacionado con el objeto destino.

Notación

Los calificadores se representan mediante un pequeño rectángulo asociado al extremo de una ruta de asociación entre el segmento final de la ruta y el símbolo de la clase calificada. El

rectángulo calificador forma parte de la ruta de asociación, no de la clase. El calificador está asociado a la clase que califica, esto es, un objeto de la clase calificada junto con un valor del calificador selecciona de modo único un conjunto de objetos de la clase destino situada en el otro extremo de la asociación.

Los atributos de calificador se enumeran dentro del cuadro calificador. Puede haber uno o más atributos en la lista. Los atributos de calificador tienen la misma notación de los atributos de clase, salvo que las expresiones de valor inicial no tienen sentido.

Opciones de presentación

Los calificadores no se pueden suprimir (proporcionan detalles esenciales cuya defecto modificaría el carácter inherente de la relación).

Las herramientas pueden utilizar una línea más fina para los rectángulos de calificador que para los rectángulos de clase, con objeto de distinguirlos claramente.

Preferiblemente, el rectángulo calificador debería ser más pequeño que el rectángulo de clase al que esté asociado, aunque esto no siempre es posible en la práctica.

Discusión

Las multiplicidades de una asociación calificada se tratan como si el objeto calificado y el calificador fueran una sola entidad, una clave compuesta. En la dirección hacia delante, la multiplicidad del extremo destino representa el número de objetos relacionados con el valor compuesto (objeto calificado + valor de calificador). En la dirección inversa, la multiplicidad describe el número de valores compuestos (objeto calificado + calificador) que están relacionados con cada objeto destino, no el número de objetos calificados relacionados con cada objeto destino. Ésta es la razón por la cual se pone el calificador en el mismo extremo de la ruta de asociación, junto al símbolo de la clase. Se puede pensar que la ruta de la asociación conecta el valor compuesto con la clase destino.

No se ha previsto especificar la multiplicidad de las relaciones no calificadas. Sin embargo, en la práctica suelen ser muchos en la dirección hacia delante. No tiene sentido tener una asociación calificada salvo que haya muchos objetos relacionados con un único objeto calificado. Para el modelado lógico, el propósito del calificador es reducir la multiplicidad a uno mediante la adición de un calificador, de tal modo que se asegure que una consulta pueda proporcionar un único valor en lugar de un conjunto de valores. La unicidad del valor del calificador suele ser frecuentemente una condición semántica crucial, que es difícil de capturar sin calificadores. Casi todas las asociaciones poseen muchas asociaciones calificadas. Hay muchos nombres que son realmente calificadores. Si un nombre es único en un determinado contexto, se trata de un calificador y el contexto debería ser identificado y modelado adecuadamente. No todos los nombres son calificadores. Los nombres de personas, por ejemplo, no son únicos. Dado que los nombres personales son ambiguos, la mayor parte de las aplicaciones de procesamiento de datos hace uso de algún tipo de número de identificación, tal como un número de cliente, un número de Seguridad Social o un número de empleado. Si una aplicación requiere la búsqueda de información o la recuperación de datos basada en claves de búsqueda, en general el modelo debería utilizar asociaciones calificadas. Todo contexto en el cual se

definan nombres o códigos de identificación para seleccionar cosas dentro de un conjunto deberá modelarse, normalmente, como una asociación calificada.

Observe que el valor del calificador es una propiedad del enlace, no del objeto destino. Considere un sistema de archivos Unix, en el cual cada directorio es una lista de entradas cuyos nombres son únicos dentro del directorio, aun cuando se pueden utilizar los mismos nombres dentro de otros directorios. Cada entrada denota un archivo, que puede ser un archivo de datos u otro directorio. Es posible tener más de una entrada que apunte a un mismo archivo. Si sucede esto, el archivo tiene varios alias. El sistema de directorios de Unix está modelado como una asociación muchos-a-uno en la cual el directorio calificado por un nombre de archivo produce un archivo. Observe que el nombre de archivo no forma parte del archivo; forma parte de la relación existente entre un directorio y un archivo. Un archivo no posee un único nombre. Puede tener múltiples nombres en otros tantos directorios (o incluso varios nombres en un mismo directorio). El nombre de archivo no es un atributo del archivo.

Una de las motivaciones más importantes para la existencia de asociaciones calificadas es la necesidad de modelar una importante situación semántica que posee una estructura de datos de implementación natural e importante. En la dirección hacia delante, una asociación calificada es una tabla de búsqueda; para un objeto calificado, cada valor del calificador produce un único objeto destino (o un valor nulo si el valor del calificador está ausente en el conjunto de valores). Las tablas de búsqueda se pueden implementar mediante estructuras de datos tales como tablas hash, árboles-b y listas ordenadas que proporcionan una eficiencia mucho mayor que las listas desordenadas. En casi todos los casos, son malos los diseños que utilizan una lista enlazada o alguna otra estructura desordenada para buscar nombres o códigos, aunque lamentablemente hay muchos programadores que las usan. El modelado de situaciones adecuadas empleando asociaciones calificadas y estructuras de datos eficientes para implementarlas resulta crucial para una buena programación.

Para un modelo lógico, tiene poco sentido tener una asociación calificada con una multiplicidad de muchos en la dirección hacia adelante, porque el calificador no añade ninguna información semántica que no pudiera mostrar un atributo de asociación. Sin embargo, en un modelo destinado al diseño de algoritmos y estructuras de datos, un calificador aporta una connotación adicional, a saber, la intención de que la selección sea eficiente. En otras palabras, una asociación calificada denota una estructura de datos indizada y optimizada para la búsqueda por valores del calificador. En este caso, puede ser útil una multiplicidad de muchos para representar un conjunto de valores al que deba poderse acceder en bloque a través de un valor de índice común, sin tener que buscar otros valores.

En general, no debe incluirse un atributo calificado como atributo de la clase destino, porque es suficiente su presencia en la asociación. En el caso de un valor índice, sin embargo, puede ser necesario tomar un valor que sea inherentemente un atributo de la clase destino y hacer que sea un valor de calificador redundante. Los valores de índice son inherentemente redundantes.

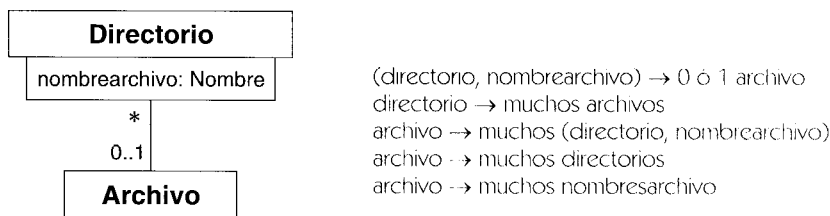


Figura 13.26 Calificador sencillo

Restricciones

Algunas situaciones complicadas no son fáciles de modelar con cualquier conjunto de relaciones no redundantes. Lo mejor es modelarlas empleando asociaciones calificadas para capturar las rutas básicas de acceso y enunciar explícitamente las restricciones adicionales. Dado que estas situaciones son poco frecuentes, estimamos que intentar incluirlas en una notación que capture directamente todas las posibles restricciones de multiplicidad no merecía la complejidad adicional.

Por ejemplo, considere un directorio en el que cada nombre de archivo identifica a un archivo único. Un archivo puede corresponder a múltiples parejas directorio-nombre de archivo. Éste es el modelo básico que se ha visto antes. El modelo se muestra en la Figura 13.26.

Ahora, sin embargo, deseamos añadir restricciones adicionales. Suponga que todo archivo debe estar en un único directorio, pero dentro de ese directorio puede tener múltiples nombres, esto es, que hay más de una forma de dar nombre al mismo archivo. Esto se podría modelar con una asociación redundante entre **Archivo** y **Directorio**, con una multiplicidad de un **Directorio** (Figura 13.27). La redundancia de las dos asociaciones se indica mediante la restricción **{igual}**, que implica que los dos elementos son el mismo pero con distintos niveles de detalle. Dado que estas asociaciones son redundantes, sólo se implementaría la asociación calificada; la otra se trataría como una restricción aplicada a sus componentes en tiempo de ejecución.

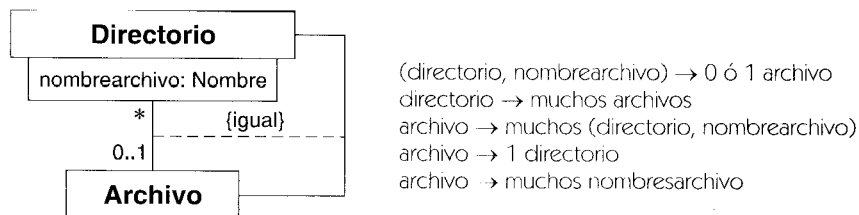


Figura 13.27 Un archivo con múltiples nombres en un directorio

Una restricción similar es que todo archivo puede aparecer en múltiples directorios pero siempre tiene el mismo nombre dondequiera que aparezca. Otros archivos podrán tener el mismo nombre pero tienen que estar en otro directorio. Esto se puede modelar haciendo que **nombearchivo** sea un atributo de **Archivo** pero restringiendo el atributo de clase y el calificador para que sean iguales (Figura 13.28). Este patrón aparece con frecuencia como índice de

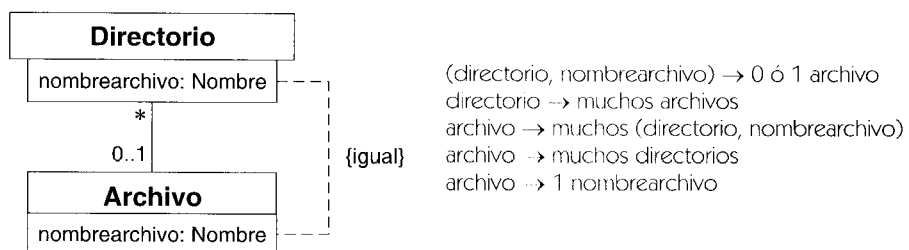


Figura 13.28 Un archivo con el mismo nombre en todos los directorios

búsqueda, aun cuando en un índice general la multiplicidad del destino calificado sería muchos. Por tanto, esta situación tiene más contenido sintáctico que un índice, que es un dispositivo de implementación.

Un tercer caso permitiría que un archivo apareciera en múltiples directorios con diferentes nombres, pero el archivo sólo podría aparecer una sola vez en cada directorio individual. Esto se podría modelar con una asociación calificada y una clase de asociación redundantes que compartieran el mismo atributo **nombreakivo** (Figura 13.29).

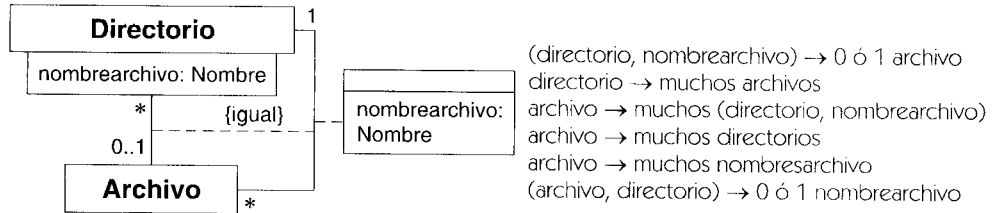


Figura 13.29 Un archivo con un nombre como máximo en cualquier directorio

Se han mostrado estos ejemplos con relaciones redundantes para ilustrar la naturaleza de las restricciones. Sin embargo, en la práctica suele resultar satisfactorio enunciar en forma de texto la restricción, mostrando gráficamente la asociación calificada.

calle¹

Partición de los grafos de actividades que se emplea para organizar las responsabilidades de las actividades. Las calles no tienen un significado fijo, pero suelen corresponder a las unidades organizativas en los modelos de negocios.

Véase también grafo de actividades.

Semántica

Los estados de actividad dentro de un grafo de actividades se pueden organizar en particiones llamadas calles como consecuencia de su notación. Las calles son agrupamientos de estados para organizar los grafos de actividades. Cada calle representa alguna partición significativa de las responsabilidades de los estados —por ejemplo, la organización de negocios responsable de un paso en un flujo de trabajo—. Se pueden utilizar en la forma que prefiera el creador del modelo. Las calles, si existen, reparten entre ellas los estados del grafo de actividades.

Cada calle posee un nombre distinto del de las otras calles. No posee ninguna semántica adicional dentro de UML pero puede tener algunas implicaciones propias del mundo real.

¹ Se alude al término empleado en natación, *swimlane*. N. del T.

Notación

Se puede dividir visualmente el diagrama de actividades en calles, cada una de las cuales está separada de las calles contiguas mediante una línea vertical continua (Figura 13.30). Cada calle representa alguna responsabilidad de alto nivel para una parte de la actividad global, que puede ser implementada eventualmente por uno o más objetos. El orden relativo de calles no posee significación semántica pero puede indicar alguna afinidad en el mundo real. Cada estado de actividad se asigna a un calle y se sitúa visualmente en su interior. Las transiciones pueden cruzar las calles; la ruta de una transición no tiene significado especial.

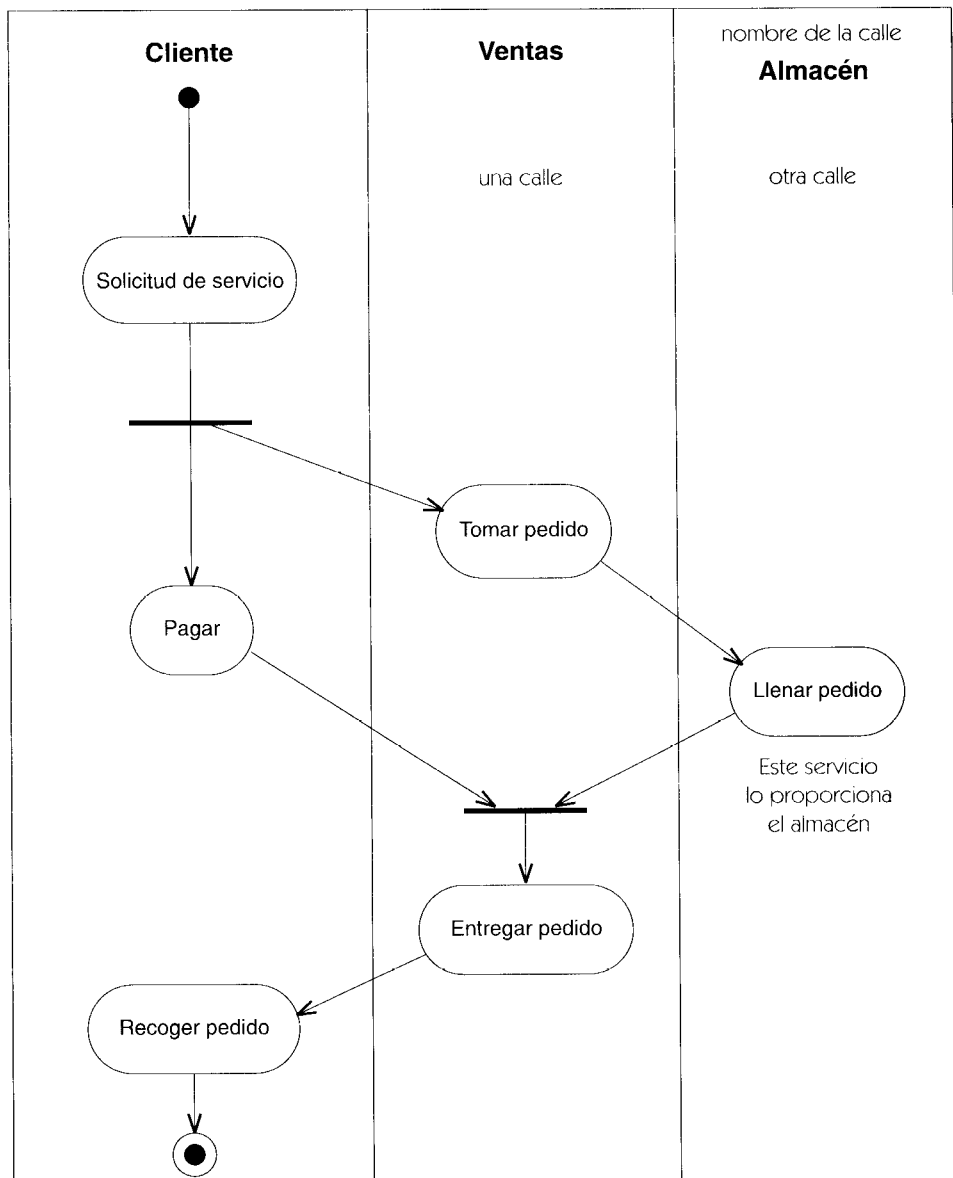


Figura 13.30 Calles de un diagrama de actividades

Como las calles no son otra cosa que particiones en categorías arbitrarias, pueden indicarse por otros medios si no resulta práctico agruparlas en regiones. Entre las posibilidades se cuenta el uso del color o utilizar simplemente valores etiquetados para mostrar la partición.

capa

Dícese de un patrón de arquitectura propio de los paquetes agrupados de un modelo, que poseen todos el mismo nivel de abstracción. Cada capa representa un mundo virtual en un determinado nivel de realidad.

característica

Una propiedad tal como una operación o atributo, que se encapsula como parte de una lista dentro de un clasificador, tal como una interfaz, una clase, o un tipo de dato.

característica de comportamiento

Elemento del modelo que expresa comportamiento dinámico, como una operación o método, y que puede ser una parte de un clasificador. La declaración de que un clasificador trata una señal, es también una característica de comportamiento.

Elementos estándar

create, destroy, leaf.

característica estructural

Característica estática de un elemento del modelo, tal como un atributo o una operación.

cardinalidad

Número de elementos de un conjunto. Es un número específico. Contrasta con multiplicidad, que es el rango de posibles cardinalidades que puede tener un conjunto.

Discusión

Obsérvese que muchos autores emplean el término *cardinalidad* de forma incorrecta queriendo hacer referencia a la multiplicidad, aunque el término cardinalidad tiene una definición matemática desde hace mucho tiempo como número, no como rango de números. Ésta es la definición que utilizamos aquí.